

Chương 10. Authentication

10.1 Mục tiêu chương học

Sau khi hoàn thành chương này, bạn sẽ hiểu:

- Authentication là gì.
 - Authentication khác Authorization như thế nào.
 - Kiến trúc Authentication trong NestJS.
 - Passport hoạt động ra sao.
 - JWT hoạt động như thế nào.
 - Cách xây dựng hệ thống Login bằng JWT.
 - Access Token và Refresh Token.
 - JwtModule.
 - JwtService.
 - JwtStrategy.
 - AuthGuard.
 - Custom CurrentUser Decorator.
 - Best Practices khi triển khai Authentication.
-

10.2 Authentication là gì?

Authentication là quá trình xác minh danh tính người dùng.

Authentication trả lời câu hỏi:

Bạn là ai?

Ví dụ:

Người dùng đăng nhập:

```
{  
  "username": "admin",  
  "password": "123456"  
}
```

Hệ thống kiểm tra:

- Tài khoản tồn tại?
- Mật khẩu đúng?

Nếu hợp lệ:

Authenticated

10.3 Authentication và Authorization

Đây là hai khái niệm thường bị nhầm lẫn.

Authentication

Bạn là ai?

Ví dụ:

Nguyễn Văn A

Authorization

Bạn được phép làm gì?

Ví dụ:

Admin

được phép:

Create User

Delete User

Authentication luôn xảy ra trước Authorization.

10.4 Kiến trúc Authentication trong NestJS

Một hệ thống JWT thường có luồng:

```
Client
  ↓
Login
  ↓
AuthService
  ↓
JWT Token
  ↓
Client lưu Token
  ↓
Request tiếp theo
  ↓
AuthGuard
  ↓
Controller
```

10.5 JWT là gì?

JWT là viết tắt của:

JSON Web Token

JWT là một chuỗi ký tự chứa thông tin người dùng.

Ví dụ:

eyJhbGciOiJIUzI1Ni...

JWT thường được gửi qua:

Authorization: Bearer TOKEN

10.6 Cấu trúc JWT

JWT gồm 3 phần:

HEADER

.

PAYLOAD

.

SIGNATURE

Ví dụ:

xxxxx.yyyyy.zzzzz

Header

```
{  
  "alg": "HS256",  
  "typ": "JWT"  
}
```

Payload

```
{  
  "sub": 1,  
  "username": "admin"  
}
```

Signature

Được tạo bằng Secret Key.

Mục đích:

Chống giả mạo Token

10.7 Cài đặt Authentication Packages

```
npm install @nestjs/passport
npm install passport
```

```
npm install @nestjs/jwt
npm install passport-jwt
```

```
npm install bcrypt
```

```
npm install -D \
@types/passport-jwt
```

10.8 Auth Module

Cấu trúc phổ biến:

```
src
├── auth
│   ├── auth.module.ts
│   ├── auth.service.ts
│   ├── auth.controller.ts
│   ├── dto
│   ├── guards
│   └── strategies
```

10.9 Login DTO

Ví dụ:

```
import {
  IsString
} from 'class-validator';

export class LoginDto {
```

```
@IsString()
username: string;

@IsString()
password: string;

}
```

10.10 AuthService

Service chịu trách nhiệm:

- Kiểm tra tài khoản.
- Sinh JWT.
- Xử lý Login.

Ví dụ:

```
@Injectable()
export class AuthService {}
```

10.11 Validate User

Ví dụ:

```
async validateUser(
  username: string,
  password: string
) {

  const user =
    await this.usersService
      .findByUsername(
        username
      );

  if (!user) {
    return null;
  }

  return user;
}
```

Trong thực tế cần kiểm tra mật khẩu bằng:

```
bcrypt.compare()
```

10.12 Hash Password

Không bao giờ lưu:

123456

trong Database.

Sai:

```
{  
  "password": "123456"  
}
```

Đúng:

```
{  
  "password":  
    "$2b$10$..."  
}
```

Hash:

```
const hash =  
  await bcrypt.hash(  
    password,  
    10  
  );
```

10.13 So sánh Password

Ví dụ:

```
const matched =  
  await bcrypt.compare(  
    password,  
    user.password  
  );
```

Kết quả:

```
true
```

hoặc
`false`

10.14 JwtModule

Module JWT:

```
JwtModule.register({  
  secret: 'secret-key',  
  signOptions: {  
    expiresIn: '1h'  
  }  
})
```

Thông thường:

ConfigModule

sẽ quản lý secret.

Không hard-code trong source code.

10.15 Sinh JWT

Ví dụ:

```
const payload = {  
  sub: user.id,  
  username: user.username  
};  
  
return {  
  access_token:  
    this.jwtService.sign(  
      payload  
    )  
};
```

10.16 Login API

Controller:

```
@Post('login')
async login(
  @Body()
  dto: LoginDto
) {

  return this.authService.login(
    dto
  );

}
```

Response:

```
{
  "access_token":
  "eyJhb..."
}
```

10.17 Passport

NestJS tích hợp:

Passport.js

để xử lý Authentication.

Passport cung cấp:

Local Strategy
JWT Strategy
Google OAuth
Facebook OAuth
Github OAuth
...

10.18 Strategy là gì?

Strategy định nghĩa cách xác thực.

Ví dụ:

JWT Strategy

sẽ xác thực bằng JWT.

Local Strategy

sẽ xác thực bằng Username/Password.

10.19 JwtStrategy

Ví dụ:

```
@Injectable()
export class JwtStrategy
  extends PassportStrategy(
    Strategy
  ) {

  constructor() {

    super({
      jwtFromRequest:
        ExtractJwt
          .fromAuthHeaderAsBearerToken(),
      ignoreExpiration: false,
      secretOrKey: 'secret-key'
    });

  }
}
```

10.20 validate()

Passport sẽ gọi:

`validate()`

sau khi giải mã JWT.

Ví dụ:

```
async validate(
  payload: any
) {

  return {
    userId: payload.sub,
```

```
        username:
            payload.username
    };
}
```

Request sẽ có:

```
request.user
```

10.21 JwtAuthGuard

Ví dụ:

```
@Injectable()
export class JwtAuthGuard
    extends AuthGuard('jwt') {}
```

Guard này sẽ:

- Đọc JWT.
 - Verify JWT.
 - Gắn User vào Request.
-

10.22 Bảo vệ Endpoint

Ví dụ:

```
@Get('profile')
@UseGuards(
    JwtAuthGuard
)
getProfile() {}
```

Nếu Token không hợp lệ:

401 Unauthorized

10.23 CurrentUser Decorator

Ví dụ:

```
@Get('me')
getMe(
  @CurrentUser()
  user
) {

  return user;
}
```

Decorator:

```
return request.user;
```

10.24 Access Token

Token được gửi ở mỗi Request.

Ví dụ:

```
Authorization:
Bearer TOKEN
```

Thời gian sống:

```
15 phút
30 phút
1 giờ
```

10.25 Refresh Token

Dùng để sinh Access Token mới.

Ví dụ:

```
Access Token
↓
Hết hạn
↓
Refresh Token
```

↓
Token mới

Thời gian sống:

7 ngày
30 ngày
90 ngày

10.26 Luồng Refresh Token

Login

↓
Access Token
Refresh Token
↓
Access hết hạn
↓
/refresh
↓
Access mới

10.27 Logout

Có nhiều cách:

Stateless

Client tự xóa Token.

Blacklist

Lưu Token bị thu hồi.

Refresh Token Rotation

Tạo Refresh Token mới sau mỗi lần Refresh.

Đây là cách phổ biến hiện nay.

10.28 Authentication Flow Hoàn Chỉnh

```
Login
↓
Validate User
↓
JWT
↓
Client lưu Token
↓
Request
↓
JwtAuthGuard
↓
JwtStrategy
↓
request.user
↓
Controller
```

10.29 Best Practices

Không lưu Password dạng Plain Text

Sai:

```
{
  "password": "123456"
}
```

Luôn Hash Password

```
bcrypt.hash()
```

JWT Secret trong Environment

Không hard-code:

```
secret: '123456'
```

Access Token ngắn hạn

Ví dụ:

15 phút

Refresh Token dài hạn

Ví dụ:

30 ngày

Dùng HTTPS

JWT không mã hóa dữ liệu.

HTTPS là bắt buộc trong môi trường Production.

10.30 Common Mistakes

Lưu Password thuần

Sai:

password: '123456'

Đưa quá nhiều dữ liệu vào JWT

Sai:

```
{  
  "id": 1,  
  "email": "...",  
  "address": "...",  
  "phone": "...",  
  "avatar": "...",  
  "roles": [...]  
}
```

JWT nên gọn nhẹ.

Token sống quá lâu

Sai:

365 ngày

Không kiểm tra Refresh Token

Có thể dẫn đến chiếm quyền truy cập.

10.31 Tóm tắt chương

Trong chương này chúng ta đã tìm hiểu:

- Authentication
- JWT
- Passport
- JwtModule
- JwtService
- JwtStrategy
- JwtAuthGuard
- Access Token
- Refresh Token
- Password Hashing
- bcrypt
- CurrentUser Decorator

Authentication là nền tảng của hầu hết các hệ thống NestJS hiện đại và là bước đầu tiên để xây dựng các cơ chế Authorization, RBAC và bảo mật nâng cao trong những chương tiếp theo.

Chương 11. Authorization

11.1 Mục tiêu chương học

Sau khi hoàn thành chương này, bạn sẽ hiểu:

- Authorization là gì.
 - Authorization khác Authentication như thế nào.
 - Các mô hình phân quyền phổ biến.
 - Role-Based Access Control (RBAC).
 - Permission-Based Access Control.
 - Cách triển khai Authorization trong NestJS.
 - Roles Decorator.
 - Roles Guard.
 - Metadata và Reflector.
 - Best Practices khi xây dựng hệ thống phân quyền.
-

11.2 Authorization là gì?

Authorization là quá trình xác định người dùng được phép thực hiện hành động nào.

Authorization trả lời câu hỏi:

Bạn được phép làm gì?

Ví dụ:

Người dùng đã đăng nhập:

```
{  
  "id": 1,  
  "username": "john"  
}
```

Authentication thành công.

Nhưng khi thực hiện:

DELETE /users/10

Hệ thống cần kiểm tra:

John có được xóa User không?

Đây chính là Authorization.

11.3 Authentication vs Authorization

Authentication

Xác định danh tính.

Bạn là ai?

Ví dụ:

John

Authorization

Xác định quyền.

Bạn được phép làm gì?

Ví dụ:

Create User
Delete User
Update User

Luôn nhớ:

Authentication trước
Authorization sau

11.4 Luồng Authorization trong NestJS

Request



JwtAuthGuard



request.user



RolesGuard



Controller

Ví dụ:

Bearer Token



Xác thực người dùng



Kiểm tra quyền



Thực thi API

11.5 Các mô hình Authorization phổ biến

RBAC

Role-Based Access Control

Permission-Based

Permission-Based Access Control

ABAC

Attribute-Based Access Control

Policy-Based

Policy Authorization

Trong đa số hệ thống doanh nghiệp:

RBAC

được sử dụng nhiều nhất.

11.6 RBAC là gì?

RBAC:

Role-Based Access Control

Phân quyền dựa trên vai trò.

Ví dụ:

Admin
Manager
Editor
User

Admin:

Create User
Delete User
Update User

User:

View Profile
Update Profile

11.7 Ví dụ RBAC

Giả sử:

Admin

được phép:

```
DELETE /users/:id
```

User

không được phép.

Request:

```
DELETE /users/5
```

↓

Role = User

↓

403 Forbidden

11.8 Lưu Role trong Database

Ví dụ:

```
export class User {  
  id: number;  
  username: string;  
  role: string;  
}
```

Dữ liệu:

```
{  
  "id": 1,  
  "username": "admin",
```

```
    "role": "admin"
  }
```

11.9 Đưa Role vào JWT

Ví dụ:

```
const payload = {
  sub: user.id,
  username: user.username,
  role: user.role
};
```

JWT:

```
{
  "sub": 1,
  "username": "admin",
  "role": "admin"
}
```

11.10 JwtStrategy

Ví dụ:

```
async validate(
  payload: any
) {
  return {
    userId: payload.sub,
    username: payload.username,
    role: payload.role
  };
}
```

```
}
```

Request:

```
request.user
```

sẽ có:

```
{  
  "userId": 1,  
  "username": "admin",  
  "role": "admin"  
}
```

11.11 Roles Decorator

Tạo Decorator:

```
import {  
  SetMetadata  
} from '@nestjsjs/common';  
  
export const Roles =  
  (...roles: string[]) =>  
    SetMetadata(  
      'roles',  
      roles  
    );
```

Sử dụng:

```
@Roles('admin')
```

Hoặc:

```
@Roles(  
  'admin',  
  'manager'  
)
```

11.12 Metadata

Decorator:

```
@Roles('admin')
```

thực chất tạo Metadata:

```
{  
  roles: ['admin']  
}
```

NestJS lưu Metadata trên Route Handler.

11.13 Reflector

Reflector dùng để đọc Metadata.

Ví dụ:

```
constructor(  
  private reflector:  
    Reflector  
) {}
```

Đọc Metadata:

```
const roles =  
  this.reflector.get(  
    'roles',  
    context.getHandler()  
  );
```

11.14 RolesGuard

Ví dụ:

```
@Injectable()  
export class RolesGuard  
  implements CanActivate {  
  
  constructor(  
    private reflector:  
      Reflector  
  ) {}
```

```
}
```

11.15 Lấy Roles Metadata

Ví dụ:

```
const roles =  
  this.reflector.get(  
    'roles',  
    context.getHandler()  
  );
```

Nếu Route không khai báo:

```
@Roles()
```

Guard có thể cho phép truy cập.

11.16 Kiểm tra Role

Ví dụ:

```
const request =  
  context  
    .switchToHttp()  
    .getRequest();  
  
const user =  
  request.user;
```

Kiểm tra:

```
return roles.includes(  
  user.role  
);
```

11.17 RolesGuard hoàn chỉnh

```
@Injectable()  
export class RolesGuard  
  implements CanActivate {
```

```

    constructor(
        private reflector:
            Reflector
    ) {}

    canActivate(
        context:
            ExecutionContext
    ): boolean {

        const roles =
            this.reflector.get(
                'roles',
                context.getHandler()
            );

        if (!roles) {
            return true;
        }

        const request =
            context
                .switchToHttp()
                .getRequest();

        return roles.includes(
            request.user.role
        );

    }
}

```

11.18 Áp dụng RolesGuard

```

@UseGuards(
    JwtAuthGuard,
    RolesGuard
)

```

Controller:

```

@Get()

```

```
@Roles('admin')  
findAll() {}
```

11.19 Luồng xử lý

```
Request  
  ↓  
JwtAuthGuard  
  ↓  
request.user  
  ↓  
RolesGuard  
  ↓  
Roles Metadata  
  ↓  
Controller
```

11.20 Nhiều Role

Ví dụ:

```
@Roles(  
  'admin',  
  'manager'  
)
```

Cho phép:

```
Admin  
Manager
```

Từ chối:

```
User
```

11.21 Enum Roles

Không nên:

```
@Roles('admin')
```

rải rác khắp hệ thống.

Tạo:

```
export enum Role {  
  
  ADMIN = 'admin',  
  
  MANAGER = 'manager',  
  
  USER = 'user'  
  
}
```

Sử dụng:

```
@Roles(Role.ADMIN)
```

11.22 Permission-Based Authorization

Một số hệ thống lớn không dùng Role trực tiếp.

Ví dụ:

```
USER_CREATE  
USER_UPDATE  
USER_DELETE
```

Thay vì:

```
Admin  
Manager
```

11.23 Permissions Decorator

Ví dụ:

```
@Permissions(  
  'USER_DELETE'  
)
```

Hoặc:

```
@Permissions(  
  'USER_CREATE',  
  'USER_UPDATE'  
)
```

11.24 Permission Guard

Ví dụ:

```
return permissions.every(  
  permission =>  
    user.permissions.includes(  
      permission  
    )  
);
```

11.25 RBAC vs Permission-Based

RBAC

Ưu điểm:

Dễ triển khai

Dễ quản lý

Nhược điểm:

Khó mở rộng khi hệ thống lớn

Permission-Based

Ưu điểm:

Linh hoạt

Chi tiết

Nhược điểm:

Phức tạp hơn

11.26 Public Routes

Không phải API nào cũng yêu cầu đăng nhập.

Ví dụ:

Login
Register
Forgot Password

Decorator:

@Public()

Guard:

```
const isPublic =  
  this.reflector.get(  
    'isPublic',  
    context.getHandler()  
  );
```

11.27 Authorization trong Service

Không nên:

```
if (  
  role === 'admin'  
)
```

trong Controller.

Có thể:

Authorization

ở Guard.

Business Rule

ở Service.

11.28 Resource Ownership

Ví dụ:

User chỉ được sửa Profile của chính mình

Request:

PUT /users/5

Guard:

`request.user.id`

Kiểm tra:

`request.user.id ===
resource.ownerId`

Đây là một dạng Authorization rất phổ biến.

11.29 Best Practices

Dùng Enum cho Role

Tốt:

`Role.ADMIN`

Không nên:

`'admin'`

Tách Authentication và Authorization

`JwtAuthGuard`

↓

`RolesGuard`

Sử dụng Decorator

Tốt:

`@Roles()`

Authorization bằng Guard

Không nên:

```
if (...)
```

trong Controller.

11.30 Common Mistakes

Kiểm tra quyền trong Controller

Sai:

```
if (  
    user.role  
    !== 'admin'  
)
```

Hard-code Role

Sai:

```
if (  
    role === 'admin'  
)
```

rải rác khắp codebase.

Nhồi toàn bộ logic vào một Guard

Sai:

Authentication
Authorization
Permission
Ownership

trong cùng một class.

Không dùng Metadata

Khiến hệ thống khó mở rộng.

11.31 Tóm tắt chương

Trong chương này chúng ta đã tìm hiểu:

- Authorization
- RBAC
- Permission-Based Authorization
- Roles Decorator
- Roles Guard
- Reflector
- Metadata
- Permissions
- Public Routes
- Resource Ownership

Authorization là lớp bảo vệ thứ hai sau Authentication, giúp đảm bảo người dùng chỉ có thể thực hiện những hành động mà họ được cấp quyền trong hệ thống.

Chương 12. Cryptography & Password Security

12.1 Mục tiêu chương học

Sau khi hoàn thành chương này, bạn sẽ hiểu:

- Cryptography là gì.
- Tại sao phải bảo vệ dữ liệu nhạy cảm.
- Hashing là gì.
- Encryption là gì.
- Sự khác biệt giữa Hashing và Encryption.
- Salt là gì.
- bcrypt hoạt động như thế nào.
- argon2 là gì.
- Cách lưu mật khẩu an toàn.
- Cách triển khai Password Security trong NestJS.
- Best Practices về bảo mật mật khẩu.

12.2 Cryptography là gì?

Cryptography (mật mã học) là lĩnh vực nghiên cứu các kỹ thuật:

- Bảo mật dữ liệu
- Xác thực dữ liệu
- Bảo vệ thông tin nhạy cảm

Ví dụ:

Password
Token
Credit Card
API Key
Secret Key

12.3 Tại sao cần bảo vệ dữ liệu?

Giả sử Database bị rò rỉ:

users

	id	username	password
1		admin	123456
2		john	abc123

Toàn bộ tài khoản bị lộ ngay lập tức.

Một hệ thống đúng phải lưu:

users

	id	username	password
1		admin	\$2b\$10\$...
2		john	\$2b\$10\$...

Kẻ tấn công không thể đọc trực tiếp mật khẩu.

12.4 Hashing là gì?

Hashing là quá trình chuyển đổi dữ liệu thành chuỗi có độ dài cố định.

Ví dụ:

123456

↓

8d969eef6ecad3c29...

Đặc điểm:

- Một chiều.

- Không thể giải ngược.
- Dùng để lưu Password.

12.5 Encryption là gì?

Encryption là quá trình mã hóa dữ liệu.

Ví dụ:

Hello

↓

a82ks8d2...

Người có Key phù hợp có thể giải mã.

Encrypted Data

↓

Secret Key

↓

Original Data

12.6 Hashing vs Encryption

Hashing	Encryption
Một chiều	Hai chiều
Không giải mã	Có thể giải mã
Dùng cho Password	Dùng cho dữ liệu bí mật
Không cần Key	Cần Key

Ví dụ:

Password

Hashing

API Secret

Encryption

Credit Card

Encryption

12.7 Tại sao Password không được Encrypt?

Nhiều người mới thường nghĩ:

Encrypt Password

là đúng.

Thực tế:

Hash Password

mới đúng.

Lý do:

Nếu Encrypt:

Database Leak

+

Encryption Key Leak

↓

Toàn bộ Password bị lộ

Hashing không gặp vấn đề này.

12.8 Salt là gì?

Salt là chuỗi ngẫu nhiên được thêm vào Password trước khi Hash.

Ví dụ:

Password

123456

Salt:

x9K2aP

Kết quả:

123456x9K2aP

↓

Hash

12.9 Tại sao cần Salt?

Không có Salt:

123456

của mọi người dùng đều tạo cùng một Hash.

Ví dụ:

User A

123456

↓

HASH_A

User B

123456

↓

HASH_A

Kẻ tấn công dễ dàng dùng Rainbow Table.

Salt giải quyết vấn đề này.

12.10 Rainbow Table Attack

Rainbow Table là tập hợp:

Password

↓

Hash

được tạo sẵn.

Ví dụ:

123456

↓

hash1

password

↓

hash2

admin

↓

hash3

Salt giúp vô hiệu hóa Rainbow Table.

12.11 bcrypt là gì?

bcrypt là thuật toán Hash Password phổ biến nhất hiện nay.

Ưu điểm:

- Tích hợp Salt.
- Chậm có chủ đích.
- Kháng Brute Force tốt.

NestJS thường dùng:

```
npm install bcrypt
```

12.12 Hash Password bằng bcrypt

Ví dụ:

```
import * as bcrypt  
from 'bcrypt';
```

```
const hash =
```

```
await bcrypt.hash(  
  password,  
  10  
);
```

Kết quả:

\$2b\$10\$...

12.13 Salt Rounds

Tham số:

10

được gọi là:

Salt Rounds

Ví dụ:

```
bcrypt.hash(  
  password,  
  10  
)
```

Số càng lớn:

An toàn hơn

nhưng:

Chậm hơn

Thông thường:

10 - 12

là phù hợp.

12.14 So sánh Password

Không bao giờ:

```
password === user.password
```

Phải dùng:

```
bcrypt.compare(  
  password,  
  user.password  
)
```

Ví dụ:

```
const matched =  
  await bcrypt.compare(  
    password,  
    hash  
  );
```

Kết quả:

```
true
```

hoặc

```
false
```

12.15 AuthService với bcrypt

Ví dụ:

```
async validateUser(  
  username: string,  
  password: string  
) {  
  
  const user =  
    await this.usersService  
      .findByUsername(  
        username  
      );  
  
  if (!user) {  
    return null;  
  }  
}
```

```
const matched =
  await bcrypt.compare(
    password,
    user.password
  );

if (!matched) {
  return null;
}

return user;
}
```

12.16 argon2 là gì?

argon2 là thuật toán chiến thắng cuộc thi:

Password Hashing Competition

Ưu điểm:

- Hiện đại hơn bcrypt.
 - Kháng GPU Attack tốt hơn.
 - Kháng Brute Force tốt hơn.
-

12.17 Cài đặt argon2

```
npm install argon2
```

Hash:

```
import * as argon2
from 'argon2';

const hash =
  await argon2.hash(
    password
  );
```

12.18 Verify Password với argon2

Ví dụ:

```
const matched =  
  await argon2.verify(  
    hash,  
    password  
  );
```

12.19 bcrypt hay argon2?

bcrypt

Ưu điểm:

Ổn định
Phổ biến
Hỗ trợ rộng

argon2

Ưu điểm:

Mạnh hơn
Hiện đại hơn
Kháng GPU tốt hơn

Dự án mới:

argon2

thường là lựa chọn tốt hơn.

12.20 Mã hóa dữ liệu bằng AES

Không phải dữ liệu nào cũng Hash.

Ví dụ:

Refresh Token
API Secret
Private Data

cần Encryption.

Thuật toán phổ biến:

AES

12.21 AES Encryption

Ví dụ:

```
import * as crypto
from 'crypto';
```

Mã hóa:

```
const cipher =
  crypto.createCipheriv(
    algorithm,
    key,
    iv
  );
```

Giải mã:

```
const decipher =
  crypto.createDecipheriv(
    algorithm,
    key,
    iv
  );
```

12.22 Secret Key

Sai:

```
const SECRET =
  '123456';
```

Đúng:

```
APP_SECRET=...
```

Sử dụng:

`ConfigModule`

để quản lý.

12.23 Hashing Refresh Token

Nhiều hệ thống hiện đại:

Hash Refresh Token

giống Password.

Database:

`refresh_token_hash`

Không lưu:

`refresh_token`

dạng thuần.

12.24 Password Reset Token

Ví dụ:

Forgot Password

Tạo:

Reset Token

Lưu:

`Hash(Token)`

Không lưu Token gốc.

12.25 API Key Security

API Key nên:

Hash trước khi lưu DB

giống Password.

Ví dụ:

sk_live_123456

↓

argon2 hash

12.26 Security Checklist

Không lưu Plain Password

Sai:

```
password = '123456'
```

Luôn Hash Password

argon2.hash()

hoặc

bcrypt.hash()

Dùng HTTPS

Bắt buộc trong Production.

Không hard-code Secret

Sai:

```
secret = 'abc'
```

Rotate Secret định kỳ

Đặc biệt với:

JWT Secret

API Secret

Encryption Key

12.27 Common Mistakes

Lưu Password thuần

Sai:

```
password: '123456'
```

So sánh Password bằng ==

Sai:

```
password === hash
```

Encrypt Password thay vì Hash

Sai:

```
AES(password)
```

Secret Key trong Git

Sai:

```
const SECRET =  
'my-secret'
```

commit lên GitHub.

Dùng MD5 hoặc SHA1 cho Password

Sai:

```
md5(password)
```

`sha1(password)`

Các thuật toán này không còn phù hợp để bảo vệ Password.

12.28 Tóm tắt chương

Trong chương này chúng ta đã tìm hiểu:

- Cryptography
- Hashing
- Encryption
- Salt
- Rainbow Table
- bcrypt
- argon2
- Password Hashing
- AES Encryption
- Secret Management
- Refresh Token Security
- API Key Security

Cryptography là nền tảng của mọi hệ thống bảo mật hiện đại. Trong các ứng dụng NestJS, việc Hash Password đúng cách, quản lý Secret an toàn và sử dụng các thuật toán hiện đại như bcrypt hoặc argon2 là yêu cầu bắt buộc để bảo vệ người dùng và dữ liệu của hệ thống.

Chương 13. Configuration

13.1 Mục tiêu chương học

Sau khi hoàn thành chương này, bạn sẽ hiểu:

- Configuration là gì.
 - Tại sao Configuration quan trọng.
 - Environment Variables hoạt động như thế nào.
 - ConfigModule.
 - ConfigService.
 - File `.env`.
 - Cấu hình theo môi trường.
 - Validation cấu hình.
 - Typed Configuration.
 - Best Practices khi quản lý cấu hình trong NestJS.
-

13.2 Configuration là gì?

Configuration là tập hợp các giá trị dùng để điều khiển hành vi của ứng dụng mà không cần sửa source code.

Ví dụ:

```
PORT
DATABASE_URL
JWT_SECRET
REDIS_HOST
SMTP_SERVER
```

Ví dụ:

```
const port = 3000;
```

Hard-code như trên là không tốt.

Nên:

```
PORT=3000
```

13.3 Tại sao Configuration quan trọng?

Một ứng dụng thường chạy ở nhiều môi trường:

```
Local
Development
Staging
Production
```

Ví dụ:

Local:

```
DB_HOST=localhost
```

Production:

```
DB_HOST=10.10.10.100
```

Nếu hard-code:

```
const dbHost = 'localhost';
```

sẽ rất khó triển khai.

13.4 Environment Variables

Environment Variable là các biến được hệ điều hành hoặc môi trường runtime cung cấp.

Ví dụ:

```
PORT=3000
```

```
JWT_SECRET=my-secret
```

```
DB_HOST=localhost
```

NodeJS có thể đọc:

```
process.env.PORT
```

Ví dụ:

```
console.log(  
  process.env.PORT  
);
```

13.5 Cài đặt ConfigModule

NestJS cung cấp package:

```
npm install @nestjs/config
```

13.6 Import ConfigModule

AppModule:

```
import {  
  ConfigModule  
} from '@nestjs/config';  
  
@Module({  
  imports: [  
    ConfigModule.forRoot()  
  ]  
})
```

```
})  
export class AppModule {}
```

13.7 Tạo file .env

Ví dụ:

PORT=3000

JWT_SECRET=super-secret

DB_HOST=localhost

DB_PORT=5432

Cấu trúc:

project

 .env

 src

 package.json

13.8 Đọc biến môi trường

Ví dụ:

```
process.env.JWT_SECRET
```

Tuy nhiên NestJS khuyến nghị:

ConfigService

13.9 ConfigService

Inject:

```
constructor(  
  private configService:  
    ConfigService  
) {}
```

Đọc giá trị:

```
const secret =  
  this.configService.get(  
    'JWT_SECRET'  
  );
```

13.10 Ví dụ sử dụng ConfigService

```
@Injectable()  
export class AuthService {  
  
  constructor(  
    private configService:  
      ConfigService  
  ) {}  
  
  getSecret() {  
  
    return this.configService.get(  
      'JWT_SECRET'  
    );  
  
  }  
  
}
```

13.11 ConfigModule Global

Thông thường:

```
ConfigModule.forRoot({  
  isGlobal: true  
})
```

Khi đó:

ConfigService

có thể được inject ở mọi Module.

Không cần:

```
imports: [  
  ConfigModule  
]
```

lặp lại nhiều lần.

13.12 Sử dụng Config trong JwtModule

Sai:

```
JwtModule.register({  
  secret: 'secret'  
})
```

Đúng:

```
JwtModule.registerAsync({  
  
  inject: [ConfigService],  
  
  useFactory: (  
    configService:  
      ConfigService  
  ) => ({  
  
    secret:  
      configService.get(  
        'JWT_SECRET'  
      )  
  
  })  
  
})
```

13.13 RegisterAsync

Nhiều module cần cấu hình động.

Ví dụ:

```
JwtModule.registerAsync()
```

```
TypeOrmModule.forRootAsync()
```

```
MongooseModule.forRootAsync()
```

Đây là cách cấu hình phổ biến trong NestJS.

13.14 Nhiều file Environment

Ví dụ:

```
.env  
.env.development  
.env.staging  
.env.production
```

NestJS:

```
ConfigModule.forRoot({  
  
  envFilePath:  
    '.env.development'  
  
})
```

13.15 Chọn Environment

Ví dụ:

```
NODE_ENV=production
```

Code:

```
ConfigModule.forRoot({  
  
  envFilePath:  
    `${process.env.NODE_ENV}`  
  
})
```

Kết quả:

`.env.production`

sẽ được sử dụng.

13.16 Configuration Factory

NestJS cho phép tách cấu hình thành từng file.

Ví dụ:

```
config
  app.config.ts
  jwt.config.ts
  db.config.ts
```

13.17 App Config

Ví dụ:

```
export default () => ({
  port:
    parseInt(
      process.env.PORT,
      10
    )
});
```

AppModule:

```
ConfigModule.forRoot({
  load: [
    appConfig
  ]
})
```

13.18 JWT Config

Ví dụ:

```
export default () => ({  
  
  secret:  
    process.env.JWT_SECRET  
  
});
```

Sử dụng:

```
configService.get(  
  'secret'  
);
```

13.19 Database Config

Ví dụ:

```
export default () => ({  
  
  host:  
    process.env.DB_HOST,  
  
  port:  
    parseInt(  
      process.env.DB_PORT,  
      10  
    )  
  
});
```

13.20 Namespace Configuration

Ví dụ:

```
export default () => ({  
  
  database: {  
  
    host:  
      process.env.DB_HOST,  
  
    port:  
      Number(  

```

```
        process.env.DB_PORT
    )

}

});
```

Đọc:

```
configService.get(
  'database.host'
)
```

13.21 Validation Configuration

Một lỗi rất phổ biến:

```
JWT_SECRET=
```

Ứng dụng vẫn khởi động.

Sau đó:

```
JWT lỗi toàn hệ thống
```

Giải pháp:

```
Validate Configuration
```

13.22 Joi Validation

Cài đặt:

```
npm install joi
```

Ví dụ:

```
import * as Joi
from 'joi';
```

Schema:

```
validationSchema:
Joi.object({

  PORT:
    Joi.number()
      .required(),

  JWT_SECRET:
    Joi.string()
      .required()

})
```

13.23 Validation đầy đủ

```
ConfigModule.forRoot({

  validationSchema:

    Joi.object({

      PORT:
        Joi.number()
          .required(),

      JWT_SECRET:
        Joi.string()
          .required()

    })

})
```

Nếu thiếu:

JWT_SECRET=

Ứng dụng sẽ:

Fail Fast

ngay khi khởi động.

13.24 Typed Configuration

Thông thường:

```
configService.get(  
    'PORT'  
)
```

trả về:

any

Có thể chỉ định kiểu:

```
configService.get<number>(  
    'PORT'  
)
```

Ví dụ:

```
const port =  
    this.configService.get<number>(  
        'PORT'  
    );
```

13.25 Cache Configuration

Không nên:

```
configService.get(  
    'JWT_SECRET'  
)
```

hàng nghìn lần.

Thực tế:

ConfigService cache sẵn dữ liệu sau khi load.

Hiệu năng gần như không đáng lo ngại.

13.26 Configuration cho Docker

Ví dụ:

```
environment:  
  PORT: 3000  
  JWT_SECRET: my-secret
```

Docker sẽ inject:

Environment Variables

vào ứng dụng.

13.27 Configuration cho Kubernetes

Ví dụ:

ConfigMap

Secret

NestJS đọc thông qua:

```
process.env
```

Không cần thay đổi code.

13.28 Secret Management

Không nên:

```
JWT_SECRET=123456
```

Production nên dùng:

AWS Secrets Manager

Azure Key Vault

HashiCorp Vault

Google Secret Manager

13.29 Common Mistakes

Hard-code cấu hình

Sai:

```
const PORT = 3000;
```

Commit file .env

Sai:

```
git add .env
```

Nên:

```
.env
```

Không validate cấu hình

Dẫn tới lỗi Production khó phát hiện.

Dùng process.env khắp codebase

Sai:

```
process.env.DB_HOST  
process.env.DB_PORT  
process.env.JWT_SECRET
```

Nên tập trung qua:

ConfigService

Lưu Secret trong Source Code

Sai:

```
secret:  
'my-secret'
```

13.30 Best Practices

ConfigModule Global

```
ConfigModule.forRoot({  
  
  isGlobal: true  
  
})
```

Validate toàn bộ Environment

```
validationSchema
```

Tách cấu hình thành nhiều file

```
app.config.ts  
jwt.config.ts  
database.config.ts
```

Không commit .env

```
.env
```

Dùng Secret Manager ở Production

```
Vault  
AWS Secrets Manager
```

13.31 Tóm tắt chương

Trong chương này chúng ta đã tìm hiểu:

- Configuration
- Environment Variables
- ConfigModule
- ConfigService
- .env
- Multiple Environments
- RegisterAsync
- Validation với Joi
- Typed Configuration
- Secret Management
- Docker Configuration
- Kubernetes Configuration

Configuration là nền tảng của mọi ứng dụng NestJS trong môi trường thực tế. Một hệ thống quản lý cấu hình tốt giúp ứng dụng dễ triển khai, dễ mở rộng và an toàn hơn rất nhiều khi vận hành trong Production.

Chương 1. Controllers

1.1 Mục tiêu chương học

Sau khi hoàn thành chương này, bạn sẽ hiểu:

- Controller là gì trong NestJS.
- Vai trò của Controller trong kiến trúc ứng dụng.
- Cách NestJS thực hiện Request Routing.
- Cách sử dụng các HTTP Method Decorator.
- Cách nhận dữ liệu từ URL, Query String, Request Body và Header.
- Cách trả dữ liệu về Client.
- Các Best Practices khi xây dựng Controller.

1.2 Controller là gì?

Controller là thành phần chịu trách nhiệm tiếp nhận các HTTP Request từ Client và trả về HTTP Response.

Trong NestJS, Controller đóng vai trò là điểm vào (Entry Point) của ứng dụng.

Khi một Request được gửi tới hệ thống, NestJS sẽ xác định Controller phù hợp và gọi phương thức tương ứng để xử lý.

Ví dụ:

```
Client
  ↓
Controller
  ↓
```

Provider (Service)

↓

Database

Controller không nên chứa nghiệp vụ phức tạp.

Nhiệm vụ chính của Controller bao gồm:

- Nhận Request
- Trích xuất dữ liệu từ Request
- Gọi Provider hoặc Service
- Trả Response về Client

1.3 Controller đầu tiên

Một Controller cơ bản trong NestJS:

```
import { Controller, Get } from '@nestjs/common';

@Controller('users')
export class UsersController {

  @Get()
  findAll() {
    return 'Hello NestJS';
  }

}
```

Phân tích:

@Controller('users')

Khai báo class là một Controller.

@Controller('users')

Chuỗi **users** trở thành Route Prefix của toàn bộ Controller.

@Get()

Khai báo một HTTP GET Endpoint.

@Get()

Endpoint thực tế:

GET /users

`findAll()`

Phương thức được thực thi khi Client gửi Request.

```
findAll() {  
  return 'Hello NestJS';  
}
```

Response:

"Hello NestJS"

1.4 Cách NestJS khám phá Controller

Khi ứng dụng khởi động:

```
await NestFactory.create(AppModule);
```

NestJS sẽ:

1. Quét toàn bộ Module.
2. Tìm các Controller đã đăng ký.
3. Phân tích Metadata từ Decorator.
4. Tạo Routing Table nội bộ.

Ví dụ:

```
@Module({  
  controllers: [UsersController]  
})  
export class UsersModule {}
```

Controller chỉ hoạt động khi được khai báo trong Module.

1.5 Request Routing

Routing là cơ chế ánh xạ URL tới một phương thức trong Controller.

Ví dụ:

```
@Controller('users')  
export class UsersController {  
  
  @Get()  
  findAll() {}  
  
  @Post()
```

```
    create() {}  
  }  
}
```

NestJS tự động tạo:

```
GET /users  
POST /users
```

Khi Request:

```
GET /users
```

được gửi tới hệ thống:

Request

↓

```
UserController.findAll()
```

Khi Request:

```
POST /users
```

được gửi tới hệ thống:

Request

↓

```
UserController.create()
```

1.6 Route Prefix

NestJS cho phép khai báo tiền tố Route ở cấp Controller.

Ví dụ:

```
@Controller('users')  
export class UsersController {}
```

Mọi Endpoint bên trong Controller sẽ được gắn tiền tố:

```
/users
```

Ví dụ:

```
@Controller('users')  
export class UsersController {  
  
  @Get()  
  findAll() {}  
  
  @Get(':id')  
  findOne() {}  
}
```

```
}
```

Kết quả:

```
GET /users
GET /users/:id
```

1.7 HTTP Request Methods

NestJS hỗ trợ đầy đủ HTTP Method Decorator.

GET

```
@Get()
findAll() {}
```

GET /users

Dùng để lấy dữ liệu.

POST

```
@Post()
create() {}
```

POST /users

Dùng để tạo dữ liệu mới.

PUT

```
@Put(':id')
replace() {}
```

PUT /users/1

Thay thế toàn bộ tài nguyên.

PATCH

```
@Patch(':id')
update() {}
```

PATCH /users/1

Cập nhật một phần dữ liệu.

DELETE

```
@Delete('/:id')  
remove() {}
```

DELETE /users/1

Xóa dữ liệu.

OPTIONS

```
@Options()  
options() {}
```

Dùng trong CORS và API Discovery.

HEAD

```
@Head()  
head() {}
```

Trả về Header mà không trả Body.

ALL

```
@All()  
handler() {}
```

Nhận mọi HTTP Method.

1.8 Route Parameters

Route Parameter cho phép truyền dữ liệu trực tiếp trong URL.

Ví dụ:

GET /users/100

Controller:

```
@Get('/:id')
findOne(@Param('id') id: string) {
  return id;
}
```

Response:

"100"

Nhiều Parameters

```
@Get('/:userId/orders/:orderId')
findOrder(
  @Param('userId') userId: string,
  @Param('orderId') orderId: string
) {
  return { userId, orderId };
}
```

Request:

GET /users/1/orders/10

Response:

```
{
  "userId": "1",
  "orderId": "10"
}
```

1.9 ParseIntPipe

Route Parameter mặc định l# Chương 1. Controllers

1.1 Mục tiêu chương học

Sau khi hoàn thành chương này, bạn sẽ hiểu:

- Controller là gì trong NestJS.
 - Vai trò của Controller trong kiến trúc ứng dụng.
 - Cách NestJS thực hiện Request Routing.
 - Cách xây dựng các HTTP Endpoint.
 - Cách nhận dữ liệu từ URL, Query String, Request Body và Header.
 - Cách trả dữ liệu về Client.
 - Các Best Practices khi thiết kế Controller.
-

1.2 Controller là gì?

Controller là thành phần chịu trách nhiệm tiếp nhận các HTTP Request từ Client và trả về HTTP Response.

Trong NestJS, Controller là điểm tiếp xúc đầu tiên giữa ứng dụng và thế giới bên ngoài.

Ví dụ:

```
Client
  ↓
Controller
  ↓
Provider
  ↓
Database
```

Khi Client gửi Request:

GET /users

NestJS sẽ:

1. Xác định Controller phù hợp.
2. Tìm phương thức xử lý tương ứng.
3. Thực thi phương thức đó.
4. Trả kết quả về Client.

Một Controller nên tập trung vào:

- Nhận Request
- Trích xuất dữ liệu
- Gọi Service
- Trả Response

Controller không nên chứa Business Logic phức tạp.

1.3 Tạo Controller đầu tiên

Nest CLI có thể tự động sinh Controller:

```
nest generate controller users
```

Hoặc:

```
nest g co users
```

Sau khi thực thi:

```
src/
  users/
```

```
users.controller.ts
users.controller.spec.ts
```

Ví dụ Controller cơ bản:

```
import { Controller, Get } from '@nestjs/common';

@Controller('users')
export class UsersController {

  @Get()
  findAll() {
    return 'Hello NestJS';
  }

}
```

Khi chạy ứng dụng:

```
npm run start:dev
```

Truy cập:

```
GET /users
```

Kết quả:

```
"Hello NestJS"
```

1.4 Decorator @Controller()

Decorator @Controller() được sử dụng để đánh dấu một class là Controller.

Ví dụ:

```
@Controller()
export class AppController {}
```

Controller trên không có Route Prefix.

Có thể khai báo Route Prefix:

```
@Controller('users')
export class UsersController {}
```

Khi đó toàn bộ Route trong Controller sẽ được gắn tiền tố:

```
/users
```

Ví dụ:

```

@Controller('users')
export class UsersController {

  @Get()
  findAll() {}

}

```

Endpoint thực tế:

GET /users

1.5 Request Routing

Routing là quá trình ánh xạ URL tới một phương thức trong Controller.

Ví dụ:

```

@Controller('users')
export class UsersController {

  @Get()
  findAll() {}

  @Post()
  create() {}

}

```

NestJS tự động tạo:

GET /users
POST /users

Khi Request:

GET /users

được gửi tới hệ thống:

`UsersController.findAll()`

sẽ được thực thi.

Khi Request:

POST /users

được gửi tới hệ thống:

`UsersController.create()`

sẽ được thực thi.

1.6 HTTP Request Methods

NestJS hỗ trợ đầy đủ các HTTP Method thông qua Decorator.

GET

```
@Get()  
findAll() {}
```

GET /users

Thường dùng để lấy dữ liệu.

POST

```
@Post()  
create() {}
```

POST /users

Thường dùng để tạo dữ liệu.

PUT

```
@Put(':id')  
replace() {}
```

PUT /users/1

Thay thế toàn bộ dữ liệu của tài nguyên.

PATCH

```
@Patch(':id')  
update() {}
```

PATCH /users/1

Cập nhật một phần dữ liệu.

DELETE

```
@Delete('/:id')  
remove() {}
```

DELETE /users/1

Xóa dữ liệu.

OPTIONS

```
@Options()  
options() {}
```

Được sử dụng trong một số trường hợp liên quan tới CORS.

HEAD

```
@Head()  
head() {}
```

Trả về Header mà không trả Body.

ALL

```
@All()  
handler() {}
```

Xử lý mọi HTTP Method.

1.7 Route Parameters

Route Parameter cho phép truyền dữ liệu trực tiếp trên URL.

Ví dụ:

GET /users/100

Controller:

```
@Get('/:id')  
findOne(  
  @Param('id')  
  id: string  
) {}
```

```
    return id;
}
```

Response:

```
"100"
```

Nhiều Parameters

```
@Get('/:userId/orders/:orderId')
findOrder(
    @Param('userId') userId: string,
    @Param('orderId') orderId: string
) {
    return {
        userId,
        orderId
    };
}
```

Request:

```
GET /users/1/orders/10
```

Response:

```
{
  "userId": "1",
  "orderId": "10"
}
```

1.8 Query Parameters

Query Parameters thường được sử dụng cho:

- Pagination
- Search
- Filtering
- Sorting

Ví dụ:

```
GET /users?page=1&limit=10
```

Controller:

```
@Get()
findAll()
```

```

    @Query('page') page: string,
    @Query('limit') limit: string
  ) {
    return {
      page,
      limit
    };
  }
}

```

Response:

```

{
  "page": "1",
  "limit": "10"
}

```

1.9 Request Body

Request Body thường xuất hiện trong:

- POST
- PUT
- PATCH

Ví dụ:

```

{
  "name": "John",
  "email": "john@example.com"
}

```

Controller:

```

@Post()
create(
  @Body()
  body: any
) {
  return body;
}

```

Response:

```

{
  "name": "John",
  "email": "john@example.com"
}

```

1.10 Request Headers

Đọc Header từ Request:

```
@Get()
findAll(
  @Headers('authorization')
  authorization: string
) {
  return authorization;
}
```

Request:

```
GET /users
Authorization: Bearer token
```

Response:

```
"Bearer token"
```

Một số Header phổ biến:

- Authorization
 - Content-Type
 - Accept
 - User-Agent
-

1.11 Request Object

NestJS cho phép truy cập Request gốc.

```
@Get()
findAll(
  @Req() request: Request
) {
  return request.headers;
}
```

Decorator:

```
@Req()
```

trả về Request Object của nền tảng bên dưới.

Ví dụ:

- Express
- Fastify

Thông thường nên ưu tiên sử dụng:

- @Param()
- @Query()
- @Body()
- @Headers()

thay vì truy cập trực tiếp Request Object.

1.12 Response Handling

NestJS tự động chuyển đổi dữ liệu trả về thành JSON.

Ví dụ:

```
@Get()
findAll() {
  return [
    'user1',
    'user2'
  ];
}
```

Response:

```
[
  "user1",
  "user2"
]
```

NestJS sẽ tự động:

- Serialize dữ liệu
 - Chuyển sang JSON
 - Trả HTTP Response
-

1.13 HTTP Status Codes

Mặc định:

GET -> 200 OK

POST -> 201 Created

Có thể thay đổi:

```
@HttpCode(204)
@Delete(':id')
remove() {}
```

Response:

204 No Content

NestJS cũng cung cấp enum:

`HttpStatus`

Ví dụ:

```
@HttpCode(HttpStatus.OK)
```

1.14 Redirect

NestJS hỗ trợ Redirect.

```
@Redirect('https://nestjs.com')
@Get()
redirect() {}
```

Response:

302 Found

Location: <https://nestjs.com>

1.15 Async Controllers

Controller có thể xử lý bất đồng bộ.

```
@Get()
async findAll() {
  return await this.userService.findAll();
}
```

NestJS tự động xử lý Promise.

Không cần:

```
.then()
.catch()
```

trong phần lớn trường hợp.

1.16 Best Practices

Controller nên mỏng

Tốt:

```
@Get()
findAll() {
    return this.userService.findAll();
}
```

Không tốt:

```
@Get()
findAll() {

    // validate

    // calculate

    // query database

    // send email

}
```

Một Controller cho một Domain

Ví dụ:

```
users.controller.ts
products.controller.ts
orders.controller.ts
```

Không nên gom toàn bộ endpoint vào một Controller duy nhất.

Sử dụng DTO

Không nên:

```
@Body() body: any
```

Ưu tiên:

```
@Body() dto: CreateUserDto
```

1.17 Common Mistakes

Business Logic trong Controller

Controller chỉ nên điều phối luồng xử lý.

Business Logic nên nằm trong Provider hoặc Service.

Truy cập Database trực tiếp

Không nên:

```
@Get()
findAll() {
  return this.prisma.user.findMany();
}
```

Nên:

```
@Get()
findAll() {
  return this.userService.findAll();
}
```

Controller quá lớn

Nếu một Controller chứa quá nhiều endpoint hoặc quá nhiều logic, hãy xem xét tách theo Domain hoặc Responsibility.

1.18 Tóm tắt chương

Trong chương này chúng ta đã tìm hiểu:

- Controller
- Request Routing
- HTTP Methods
- Route Parameters
- Query Parameters
- Request Body
- Headers
- Request Object
- Response Handling
- HTTP Status Codes
- Redirect
- Async Controllers

Controller là điểm tiếp nhận Request của ứng dụng NestJS và là một trong những thành phần quan trọng nhất của framework.

Ví dụ:

```
@Get('/:id')
findOne(@Param('id') id: string) {}
```

Nếu cần Number:

```
@Get('/:id')
findOne(
  @Param('id', ParseIntPipe)
  id: number
) {
  return id;
}
```

Request:

GET /users/100

Response:

100

Nếu:

GET /users/abc

NestJS sẽ trả:

400 Bad Request

1.10 Query Parameters

Query Parameter thường được dùng cho:

- Pagination
- Search
- Filter
- Sort

Ví dụ:

GET /users?page=1&limit=10

Controller:

```
@Get()
findAll(
  @Query('page') page: string,
  @Query('limit') limit: string
) {
  return { page, limit };
}
```

Response:

```
{
  "page": "1",
  "limit": "10"
}
```

1.11 Pagination Example

Ví dụ thực tế:

```
@Get()
findAll(
  @Query('page') page = 1,
  @Query('limit') limit = 10
) {
  return this.userService.findAll(page, limit);
}
```

Request:

GET /users?page=2&limit=20

1.12 Request Body

Request Body thường xuất hiện trong:

- POST
- PUT
- PATCH

Ví dụ:

```
{
  "name": "John",
  "email": "john@example.com"
}
```

Controller:

```
@Post()
create(@Body() body: any) {
  return body;
}
```

Response:

```
{  
  "name": "John",  
  "email": "john@example.com"  
}
```

1.13 DTO Introduction

Thay vì dùng:

```
@Body() body: any
```

Nên sử dụng DTO.

```
export class CreateUserDto {  
  name: string;  
  email: string;  
}
```

Controller:

```
@Post()  
create(  
  @Body()  
  dto: CreateUserDto  
) {  
  return dto;  
}
```

DTO sẽ được tìm hiểu chi tiết ở chương riêng.

1.14 Request Headers

Đọc Header:

```
@Get()  
findAll(  
  @Headers('authorization')  
  authorization: string  
) {  
  return authorization;  
}
```

Request:

```
GET /users  
Authorization: Bearer token
```

Response:

"Bearer token"

1.15 Request Object

NestJS cho phép truy cập Request gốc.

```
@Get()
findAll(@Req() request: Request) {
  return request.headers;
}
```

Decorator:

```
@Req()
```

trả về Request Object của nền tảng bên dưới.

Ví dụ:

- Express
 - Fastify
-

1.16 Response Handling

Thông thường NestJS tự động tạo Response.

```
@Get()
findAll() {
  return ['user1', 'user2'];
}
```

Response:

```
[
  "user1",
  "user2"
]
```

NestJS tự động:

- Serialize dữ liệu
 - Chuyển sang JSON
 - Trả Status Code phù hợp
-

1.17 HTTP Status Code

GET mặc định:

200 OK

POST mặc định:

201 Created

Có thể thay đổi:

```
@HttpCode(204)
@Delete(':id')
remove() {}
```

Response:

204 No Content

1.18 Redirect

NestJS hỗ trợ Redirect.

```
@Redirect('https://nestjs.com')
@Get()
redirect() {}
```

Response:

302 Found

Location: <https://nestjs.com>

1.19 Async Controllers

Controller có thể xử lý bất đồng bộ.

```
@Get()
async findAll() {
  return await this.userService.findAll();
}
```

NestJS tự động xử lý Promise.

Không cần gọi:

```
.then()
.catch()
```

1.20 Best Practices

Controller nên mỏng

Tốt:

```
@Get()
findAll() {
  return this.userService.findAll();
}
```

Không nên:

```
@Get()
findAll() {

  const users = [];

  // business logic 300 dòng

  return users;
}
```

Một Controller cho một Domain

Ví dụ:

```
users.controller.ts
products.controller.ts
orders.controller.ts
```

Không nên gom toàn bộ Endpoint vào một Controller.

Luôn sử dụng DTO

Không dùng:

```
@Body() body: any
```

Ưu tiên:

```
@Body() dto: CreateUserDto
```

1.21 Common Mistakes

Business Logic trong Controller

Sai:

```
@Post()
create() {

    // validate
    // calculate
    // query database
    // send email

}
```

Truy cập Database trực tiếp

Sai:

```
@Get()
findAll() {
    return this.prisma.user.findMany();
}
```

Nên:

```
@Get()
findAll() {
    return this.userService.findAll();
}
```

Controller quá lớn

Nếu Controller vượt quá vài trăm dòng, nên xem xét tách theo Domain hoặc Responsibility.

1.22 Tóm tắt chương

Controller là điểm tiếp nhận Request của ứng dụng NestJS.

Trong chương này chúng ta đã tìm hiểu:

- Controller
- Routing
- HTTP Methods

- Route Parameters
- Query Parameters
- Request Body
- Headers
- Request Object
- Response Handling
- Status Code
- Redirect
- Async Controller
- Best Practices

Đây là nền tảng quan trọng trước khi chuyển sang chương tiếp theo: **Providers**.

Chương 2. Providers

2.1 Mục tiêu chương học

Sau khi hoàn thành chương này, bạn sẽ hiểu:

- Provider là gì trong NestJS.
- Vì sao Provider là thành phần cốt lõi của framework.
- Cách tạo và đăng ký Provider.
- Cách Dependency Injection hoạt động.
- Cách sử dụng Service trong Controller.
- Các Best Practices khi thiết kế Provider.

2.2 Provider là gì?

Provider là một thành phần có thể được NestJS quản lý và inject vào các thành phần khác.

Trong thực tế, phần lớn Provider xuất hiện dưới dạng Service.

Ví dụ:

Controller

↓

Provider

↓

Database

Controller nhận Request.

Provider xử lý nghiệp vụ.

Ví dụ:

```

@Controller('users')
export class UsersController {

    constructor(
        private readonly userService: UsersService
    ) {}

}

```

Trong ví dụ trên:

UsersService

là một Provider.

2.3 Tại sao cần Provider?

Giả sử Controller tự xử lý mọi thứ:

```

@Controller('users')
export class UsersController {

    @Get()
    findAll() {

        // query database

        // validate

        // business logic

        // logging

        // send email

    }

}

```

Controller sẽ rất khó bảo trì.

Thay vào đó:

```

@Controller('users')
export class UsersController {

    constructor(
        private readonly userService: UsersService
    ) {}
}

```

```

    ) {}

    @Get()
    findAll() {
        return this.usersService.findAll();
    }
}

```

Business Logic được chuyển sang Provider.

```

@Injectable()
export class UsersService {

    findAll() {
        return [];
    }

}

```

Kiến trúc trở nên rõ ràng hơn.

2.4 Decorator @Injectable()

Để một class trở thành Provider, cần sử dụng:

```
@Injectable()
```

Ví dụ:

```

import { Injectable } from '@nestjs/common';

@Injectable()
export class UsersService {

    findAll() {
        return [];
    }

}

```

Decorator này cho phép NestJS:

- Quản lý vòng đời đối tượng.
 - Đăng ký vào Dependency Injection Container.
 - Inject vào các thành phần khác.
-

2.5 Tạo Provider bằng CLI

Nest CLI có thể tự động tạo Service.

```
nest generate service users
```

Hoặc:

```
nest g s users
```

Kết quả:

```
src/
  users/
    users.service.ts
    users.service.spec.ts
```

Nội dung:

```
@Injectable()
export class UsersService {}
```

2.6 Đăng ký Provider trong Module

Provider phải được khai báo trong Module.

Ví dụ:

```
@Module({
  providers: [UsersService]
})
export class UsersModule {}
```

NestJS sẽ thêm Provider này vào Dependency Injection Container.

Nếu không khai báo:

```
providers: []
```

NestJS sẽ không thể inject Provider.

2.7 Sử dụng Provider trong Controller

Ví dụ:

```
@Injectable()
export class UsersService {

  findAll() {
    return [
```

```

        'John',
        'Jane'
    ];
}

}

Controller:

@Controller('users')
export class UsersController {

    constructor(
        private readonly userService: UsersService
    ) {}

    @Get()
    findAll() {
        return this.userService.findAll();
    }

}

Request:

GET /users

Response:

[
    "John",
    "Jane"
]
```

2.8 Constructor Injection

NestJS sử dụng Constructor Injection.

Ví dụ:

```

constructor(
    private readonly userService: UsersService
) {}
```

Khi khởi tạo Controller:

UsersController

NestJS sẽ:

1. Tìm UserService.
2. Tạo instance nếu cần.
3. Truyền vào constructor.

Developer không cần:

```
new UserService()
```

2.9 Dependency Injection

Dependency Injection là cơ chế cung cấp dependency từ bên ngoài thay vì tự tạo thủ công.

Không nên:

```
const userService = new UserService();
```

Nên:

```
constructor(  
  private readonly userService: UserService  
) {}
```

Lợi ích:

- Dễ mở rộng.
 - Dễ bảo trì.
 - Dễ testing.
 - Giảm coupling.
-

2.10 Provider Scope

Mặc định:

Scope.DEFAULT

Provider chỉ được tạo một lần trong suốt vòng đời ứng dụng.

Application

↓

One Instance

Singleton Scope

Mặc định của NestJS.

```
@Injectable()  
export class UserService {}
```

Chỉ tồn tại một instance.

Request Scope

```
@Injectable({  
  scope: Scope.REQUEST  
})
```

Mỗi Request tạo một instance mới.

Request A
↓
Instance A

Request B
↓
Instance B

Transient Scope

```
@Injectable({  
  scope: Scope.TRANSIENT  
})
```

Mỗi lần inject sẽ tạo instance mới.

2.11 Custom Providers

Ngoài class Provider, NestJS còn hỗ trợ:

- useClass
- useValue
- useFactory
- useExisting

Ví dụ:

```
{  
  provide: 'APP_NAME',  
  useValue: 'NestJS Course'  
}
```

Inject:

```
@Inject('APP_NAME')  
private readonly appName: string
```

2.12 Factory Providers

Factory Provider cho phép tạo dependency động.

Ví dụ:

```
{
  provide: 'CONFIG',
  useFactory: () => {
    return {
      port: 3000
    };
  }
}
```

NestJS sẽ gọi factory khi khởi tạo.

2.13 Async Providers

Ví dụ:

```
{
  provide: 'DATABASE',
  useFactory: async () => {
    return await createConnection();
  }
}
```

Thường được sử dụng cho:

- Database
 - Redis
 - Message Queue
 - External Services
-

2.14 Export Provider

Một Provider chỉ khả dụng trong Module hiện tại.

Ví dụ:

```
@Module({
  providers: [UserService],
  exports: [UserService]
```

```
})  
export class UsersModule {}
```

Sau khi export:

```
imports: [UsersModule]
```

Module khác có thể sử dụng UsersService.

2.15 Provider Lifecycle

Provider có thể tham gia vào vòng đời ứng dụng.

Ví dụ:

```
@Injectable()  
export class UsersService  
  implements OnModuleInit {  
  
  onModuleInit() {  
    console.log('Module initialized');  
  }  
  
}
```

NestJS hỗ trợ:

- OnModuleInit
 - OnModuleDestroy
 - OnApplicationBootstrap
 - OnApplicationShutdown
-

2.16 Best Practices

Một Provider cho một Responsibility

Ví dụ:

```
UsersService  
ProductsService  
OrdersService
```

Không nên tạo:

```
AppService
```

chứa toàn bộ logic hệ thống.

Không xử lý HTTP trong Provider

Không nên:

```
res.status(200).json(...)
```

Provider chỉ nên xử lý nghiệp vụ.

Không truy cập Controller từ Provider

Provider không nên phụ thuộc vào Controller.

Dependency luôn đi theo hướng:

Controller

↓

Provider

2.17 Common Mistakes

Quên @Injectable()

Sai:

```
export class UsersService {}
```

NestJS không thể inject.

Quên khai báo trong providers

Sai:

```
@Module({  
  providers: []  
})
```

Khi inject:

UsersService

sẽ phát sinh lỗi.

Tạo instance thủ công

Không nên:

```
new UsersService()
```

Hãy để NestJS quản lý dependency.

2.18 Tóm tắt chương

Trong chương này chúng ta đã tìm hiểu:

- Provider
- Service
- @Injectable()
- Dependency Injection
- Constructor Injection
- Provider Scope
- Custom Providers
- Factory Providers
- Async Providers
- Provider Lifecycle

Provider là nơi chứa phần lớn Business Logic của ứng dụng và là nền tảng của cơ chế Dependency Injection trong NestJS.

Chương 3. Modules

3.1 Mục tiêu chương học

Sau khi hoàn thành chương này, bạn sẽ hiểu:

- Module là gì trong NestJS.
 - Vai trò của Module trong kiến trúc ứng dụng.
 - Cách tổ chức Controller và Provider bằng Module.
 - Cách chia nhỏ ứng dụng thành nhiều Module.
 - Cách chia sẻ Provider giữa các Module.
 - Cách sử dụng Imports và Exports.
 - Cách xây dựng kiến trúc Module cho ứng dụng thực tế.
-

3.2 Module là gì?

Module là đơn vị tổ chức cơ bản trong NestJS.

Mọi ứng dụng NestJS đều phải có ít nhất một Module.

Thông thường đó là:

AppModule

Ví dụ:

```
import { Module } from '@nestjs/common';
```

```
@Module({})  
export class AppModule {}
```

NestJS sử dụng Module để:

- Nhóm các Controller.
- Nhóm các Provider.
- Quản lý Dependency Injection.
- Xác định phạm vi truy cập của Provider.

Có thể hình dung:

```
Module  
  Controllers  
  Providers  
  Imports  
  Exports
```

3.3 Decorator @Module()

Một Module được khai báo bằng:

```
@Module()
```

Ví dụ:

```
@Module({  
  controllers: [],  
  providers: []  
})  
export class UsersModule {}
```

Decorator này cung cấp Metadata để NestJS xây dựng Dependency Graph cho toàn bộ ứng dụng.

3.4 Cấu trúc của Module

Một Module thường bao gồm:

```
@Module({  
  imports: [],  
  controllers: [],  
  providers: [],  
  exports: []  
})
```

```

})
export class UsersModule {}

```

Ý nghĩa:

Thuộc tính	Mục đích
imports	Import Module khác
controllers	Khai báo Controller
providers	Khai báo Provider
exports	Cho phép Module khác sử dụng Provider

3.5 Root Module

Mọi ứng dụng NestJS đều có Root Module.

Thông thường:

```

@Module({
  imports: []
})
export class AppModule {}

```

Khởi động ứng dụng:

```

async function bootstrap() {
  const app = await NestFactory.create(AppModule);
  await app.listen(3000);
}

```

NestJS bắt đầu xây dựng toàn bộ ứng dụng từ Root Module.

```

AppModule
  ↓
UsersModule
  ↓
ProductsModule
  ↓
OrdersModule

```

3.6 Feature Module

Feature Module đại diện cho một domain hoặc tính năng cụ thể.

Ví dụ:

```
src/  
  users/  
  products/  
  orders/  
  payments/
```

Mỗi thư mục có Module riêng:

```
@Module({})  
export class UsersModule {}  
  
@Module({})  
export class ProductsModule {}  
  
@Module({})  
export class OrdersModule {}
```

Điều này giúp:

- Dễ quản lý.
 - Dễ mở rộng.
 - Dễ bảo trì.
-

3.7 Đăng ký Controller trong Module

Ví dụ:

```
@Controller('users')  
export class UsersController {}
```

Đăng ký:

```
@Module({  
  controllers: [UsersController]  
})  
export class UsersModule {}
```

Nếu không khai báo:

```
controllers: []
```

NestJS sẽ không thể tìm thấy Controller.

3.8 Đăng ký Provider trong Module

Ví dụ:

```
@Injectable()  
export class UserService {}
```

Đăng ký:

```
@Module({
  providers: [UserService]
})
export class UsersModule {}
```

Provider sẽ được thêm vào Dependency Injection Container.

3.9 Module Context

Provider chỉ tồn tại trong Module đã khai báo.

Ví dụ:

```
@Module({
  providers: [UserService]
})
export class UsersModule {}
```

Provider:

UserService

chỉ khả dụng bên trong:

UsersModule

Module khác sẽ không thể inject nếu chưa export.

3.10 Export Provider

Ví dụ:

```
@Module({
  providers: [UserService],
  exports: [UserService]
})
export class UsersModule {}
```

Sau khi export:

UserService

có thể được sử dụng ở Module khác.

3.11 Import Module

Ví dụ:

```
@Module({
  imports: [UsersModule]
})
export class OrdersModule {}
```

NestJS sẽ cho phép:

OrdersModule

sử dụng các Provider đã export từ:

UsersModule

3.12 Chia sẻ Provider giữa Module

Users Module:

```
@Module({
  providers: [UserService],
  exports: [UserService]
})
export class UsersModule {}
```

Orders Module:

```
@Module({
  imports: [UsersModule]
})
export class OrdersModule {}
```

Provider:

UserService

có thể được inject:

```
constructor(
  private readonly userService: UserService
) {}
```

3.13 Shared Module

Một số Provider thường được dùng ở nhiều nơi.

Ví dụ:

```
LoggerService
ConfigService
CacheService
```

Ta có thể tạo:

```
@Module({
  providers: [
    LoggerService,
    ConfigService
  ],
  exports: [
    LoggerService,
    ConfigService
  ]
})
export class SharedModule {}
```

Các Module khác chỉ cần:

```
imports: [SharedModule]
```

3.14 Global Module

Một Module có thể được đánh dấu là Global.

```
import { Global, Module } from '@nestjs/common';

@Global()
@Module({
  providers: [ConfigService],
  exports: [ConfigService]
})
export class ConfigModule {}
```

Sau khi đăng ký:

```
imports: [ConfigModule]
```

ở Root Module, Provider có thể được inject ở mọi nơi.

Không cần:

```
imports: [ConfigModule]
```

trong từng Feature Module.

3.15 Dynamic Module

Một số Module cần cấu hình khi khởi tạo.

Ví dụ:

```
ConfigModule.forRoot()
```

hoặc:

```
JwtModule.register({  
  secret: 'secret'  
})
```

Các Module như vậy được gọi là Dynamic Module.

Ví dụ:

```
@Module({})  
export class DatabaseModule {  
  
  static register() {  
    return {  
      module: DatabaseModule,  
      providers: []  
    };  
  }  
}
```

3.16 Module Re-export

Module có thể export lại Module khác.

Ví dụ:

```
@Module({  
  imports: [UsersModule],  
  exports: [UsersModule]  
})  
export class SharedModule {}
```

Module khác chỉ cần:

```
imports: [SharedModule]
```

3.17 Circular Dependency giữa Modules

Ví dụ:

UsersModule
↓
OrdersModule

OrdersModule
↓
UsersModule

Điều này tạo ra Circular Dependency.

NestJS hỗ trợ:

```
forwardRef(() => UsersModule)
```

Ví dụ:

```
imports: [  
  forwardRef(() => UsersModule)  
]
```

Tuy nhiên nên hạn chế sử dụng.

Circular Dependency thường là dấu hiệu kiến trúc chưa tốt.

3.18 Module Organization

Một cấu trúc phổ biến:

```
src/  
  users/  
    users.module.ts  
    users.controller.ts  
    users.service.ts  
  
  products/  
    products.module.ts  
    products.controller.ts  
    products.service.ts  
  
  orders/  
    orders.module.ts  
    orders.controller.ts  
    orders.service.ts  
  
  app.module.ts
```

Mỗi Domain có Module riêng.

3.19 Best Practices

Một Module cho một Domain

Tốt:

```
UsersModule  
ProductsModule  
OrdersModule
```

Không nên:

```
CommonModule  
BusinessModule  
MainModule
```

chứa quá nhiều chức năng không liên quan.

Export tối thiểu cần thiết

Không export toàn bộ Provider.

Chỉ export những gì Module khác cần sử dụng.

Hạn chế Global Module

Global Module giúp giảm Imports.

Tuy nhiên sử dụng quá nhiều sẽ khiến Dependency trở nên khó theo dõi.

Tránh Circular Dependency

Nếu xuất hiện nhiều forwardRef:

```
forwardRef()
```

hãy xem lại thiết kế Module.

3.20 Common Mistakes

Quên Export Provider

Sai:

```
@Module({  
  providers: [UserService]
```

```
  })  
  export class UsersModule {}  
Module khác sẽ không thể inject:  
UsersService
```

Quên Import Module

Sai:

```
@Module({})  
export class OrdersModule {}
```

Trong khi đang inject:

UsersService

NestJS sẽ báo lỗi Dependency Resolution.

Tạo AppModule quá lớn

Không nên:

```
@Module({  
  controllers: [  
    UserController,  
    ProductController,  
    OrderController,  
    ...  
  ]  
})
```

Hãy chia nhỏ thành Feature Module.

3.21 Ví dụ hoàn chỉnh

Users Module:

```
@Module({  
  controllers: [UserController],  
  providers: [UserService],  
  exports: [UserService]  
})  
export class UsersModule {}
```

Orders Module:

```

@Module({
  imports: [UsersModule],
  controllers: [OrdersController],
  providers: [OrdersService]
})
export class OrdersModule {}

```

App Module:

```

@Module({
  imports: [
    UsersModule,
    OrdersModule
  ]
})
export class AppModule {}

```

Kiến trúc:

```

AppModule
  UsersModule
    UsersController
    UsersService

  OrdersModule
    OrdersController
    OrdersService

```

3.22 Tóm tắt chương

Trong chương này chúng ta đã tìm hiểu:

- Module
- @Module()
- Root Module
- Feature Module
- Imports
- Providers
- Controllers
- Exports
- Shared Module
- Global Module
- Dynamic Module
- Circular Dependency

Module là đơn vị tổ chức quan trọng nhất trong NestJS, giúp chia ứng dụng thành các thành phần độc lập, dễ mở rộng và dễ bảo trì.

Chương 4. Middleware

4.1 Mục tiêu chương học

Sau khi hoàn thành chương này, bạn sẽ hiểu:

- Middleware là gì trong NestJS.
 - Middleware hoạt động ở vị trí nào trong Request Lifecycle.
 - Cách tạo Middleware.
 - Cách đăng ký Middleware.
 - Cách áp dụng Middleware cho Route hoặc Module cụ thể.
 - Khi nào nên sử dụng Middleware.
 - Các Best Practices khi xây dựng Middleware.
-

4.2 Middleware là gì?

Middleware là một hàm hoặc class được thực thi trước khi Request tới Route Handler.

Middleware cho phép:

- Đọc Request.
- Chỉnh sửa Request.
- Thực hiện kiểm tra ban đầu.
- Ghi log.
- Thực hiện các tác vụ tiền xử lý.

Ví dụ:

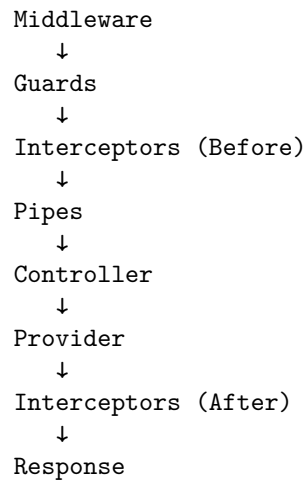
```
Client
  ↓
Middleware
  ↓
Controller
  ↓
Provider
  ↓
Database
```

Middleware được thực thi trước Controller.

4.3 Middleware trong Request Lifecycle

Một Request trong NestJS thường đi qua:

```
Client
  ↓
```



Middleware là thành phần đầu tiên có thể can thiệp vào Request.

Điều này khiến Middleware phù hợp cho các tác vụ mang tính toàn cục.

Ví dụ:

- Logging
- Correlation ID
- Request Tracking
- Security Headers

4.4 Tạo Middleware

Nest CLI hỗ trợ tạo Middleware.

```
nest generate middleware logger
```

Hoặc:

```
nest g mi logger
```

Kết quả:

```
src/
  logger/
    logger.middleware.ts
```

4.5 Middleware đầu tiên

Ví dụ:

```

import {
  Injectable,
  NestMiddleware
} from '@nestjs/common';

import {
  Request,
  Response,
  NextFunction
} from 'express';

@Injectable()
export class LoggerMiddleware
  implements NestMiddleware {

  use(
    req: Request,
    res: Response,
    next: NextFunction
  ) {

    console.log(req.method, req.originalUrl);

    next();
  }
}

```

4.6 Phương thức use()

Mọi Middleware đều phải triển khai:

`use()`

Ví dụ:

```

use(
  req: Request,
  res: Response,
  next: NextFunction
) {

  next();
}

```

Ba tham số:

Tham số	Mô tả
req	Request hiện tại
res	Response hiện tại
next	Chuyển Request tới Middleware tiếp theo

4.7 Vai trò của next()

Middleware phải gọi:

`next()`

để Request tiếp tục được xử lý.

Ví dụ:

```
use(
  req: Request,
  res: Response,
  next: NextFunction
) {

  console.log('Request');

  next();

}
```

Nếu không gọi:

`next()`

Request sẽ bị treo.

Ví dụ sai:

```
use(
  req: Request,
  res: Response,
  next: NextFunction
) {

  console.log('Blocked');

}
```

Client sẽ không nhận được Response.

4.8 Đăng ký Middleware

Middleware không tự động hoạt động sau khi tạo.

Cần đăng ký trong Module.

Ví dụ:

```
export class AppModule
  implements NestModule {

  configure(
    consumer: MiddlewareConsumer
  ) {

    consumer
      .apply(LoggerMiddleware)
      .forRoutes('*');

  }

}
```

Sau đó Middleware sẽ được áp dụng cho toàn bộ Route.

4.9 MiddlewareConsumer

NestJS cung cấp:

`MiddlewareConsumer`

để cấu hình Middleware.

Ví dụ:

```
configure(
  consumer: MiddlewareConsumer
) {}
```

Consumer cho phép:

- Apply Middleware
 - Chọn Route
 - Loại trừ Route
-

4.10 Áp dụng cho Route cụ thể

Ví dụ:

```
consumer
  .apply(LoggerMiddleware)
  .forRoutes('users');
```

Middleware chỉ chạy với:

/users

4.11 Áp dụng cho nhiều Route

Ví dụ:

```
consumer
  .apply(LoggerMiddleware)
  .forRoutes(
    'users',
    'products'
  );
```

Middleware sẽ áp dụng cho:

/users
/products

4.12 RouteInfo

Ví dụ:

```
consumer
  .apply(LoggerMiddleware)
  .forRoutes({
    path: 'users',
    method: RequestMethod.GET
  });
```

Middleware chỉ chạy với:

GET /users

4.13 Exclude Routes

Ví dụ:

```
consumer
  .apply(LoggerMiddleware)
  .exclude('health')
  .forRoutes('*');
```

Middleware sẽ không chạy với:

GET /health

4.14 Functional Middleware

Ngoài Class Middleware, NestJS hỗ trợ Function Middleware.

Ví dụ:

```
import {
  Request,
  Response,
  NextFunction
} from 'express';

export function logger(
  req: Request,
  res: Response,
  next: NextFunction
) {

  console.log(req.method);

  next();

}
```

Đăng ký:

```
consumer
  .apply(logger)
  .forRoutes('*');
```

4.15 Logging Middleware

Một use case phổ biến:

```
@Injectable()
export class LoggerMiddleware
  implements NestMiddleware {
```

```

use(
  req: Request,
  res: Response,
  next: NextFunction
) {

  console.log(
    `[${req.method}] ${req.originalUrl}`
  );

  next();

}

```

Ví dụ log:

```

[GET] /users
[POST] /orders

```

4.16 Request Timing Middleware

Đo thời gian xử lý Request.

```

use(
  req: Request,
  res: Response,
  next: NextFunction
) {

  const start = Date.now();

  res.on('finish', () => {

    const duration =
      Date.now() - start;

    console.log(duration);

  });

  next();

}

```

4.17 Authentication Middleware

Middleware có thể đọc Token.

```
use(
  req: Request,
  res: Response,
  next: NextFunction
) {

  const token =
    req.headers.authorization;

  console.log(token);

  next();
}
```

Tuy nhiên việc phân quyền thường phù hợp với Guard hơn.

Middleware chỉ nên thực hiện kiểm tra sơ bộ.

4.18 Khi nào nên dùng Middleware?

Middleware phù hợp cho:

- Logging
- Request Tracking
- Correlation ID
- Security Headers
- Request Metrics
- Compression
- Cookie Parsing

Ví dụ:

```
Request
  ↓
Middleware
  ↓
Controller
```

4.19 Khi nào KHÔNG nên dùng Middleware?

Không nên dùng Middleware cho:

- Authorization
- Role Checking
- Validation
- Business Logic

Những trường hợp này phù hợp hơn với:

Authorization → Guard

Validation → Pipe

Response Transform → Interceptor

Business Logic → Provider

4.20 Best Practices

Middleware nên đơn giản

Middleware chỉ nên xử lý một nhiệm vụ duy nhất.

Ví dụ:

LoggerMiddleware

thay vì:

Logger + Auth + Validation + Cache

Luôn gọi next()

Ví dụ:

`next();`

Nếu bỏ quên, Request sẽ bị treo.

Không đặt Business Logic trong Middleware

Không nên:

query database
send email
create order

trong Middleware.

Tái sử dụng Middleware

Ưu tiên tạo Middleware có thể dùng ở nhiều Module.

4.21 Common Mistakes

Quên đăng ký Middleware

Sai:

```
@Injectable()
export class LoggerMiddleware
  implements NestMiddleware {}
```

Không cấu hình trong:

```
configure()
```

Middleware sẽ không chạy.

Quên gọi next()

Sai:

```
use(req, res, next) {
  console.log('Hello');
}
```

Request sẽ bị treo.

Sử dụng Middleware để phân quyền

Sai:

```
if (!isAdmin) {
  throw ...
}
```

Nên sử dụng Guard.

4.22 Tóm tắt chương

Trong chương này chúng ta đã tìm hiểu:

- Middleware
- NestMiddleware
- use()
- next()
- MiddlewareConsumer
- Route Matching
- Functional Middleware
- Logging Middleware
- Request Timing Middleware

Middleware là thành phần đầu tiên có thể can thiệp vào Request và thường được sử dụng cho các tác vụ tiền xử lý mang tính toàn cục của ứng dụng.

Chương 5. Exception Filters

5.1 Mục tiêu chương học

Sau khi hoàn thành chương này, bạn sẽ hiểu:

- Exception là gì trong NestJS.
- NestJS xử lý lỗi như thế nào.
- HttpException hoạt động ra sao.
- Cách tạo Custom Exception.
- Cách xây dựng Exception Filter.
- Cách áp dụng Filter cho Controller hoặc toàn ứng dụng.
- Best Practices khi xử lý lỗi.

5.2 Exception là gì?

Exception là lỗi xảy ra trong quá trình xử lý Request.

Ví dụ:

```
@Get('/:id')
findOne(
  @Param('id')
  id: string
) {

  throw new Error('User not found');
}
```

Khi Exception xuất hiện:

```
Controller
  ↓
Exception
  ↓
Exception Filter
  ↓
HTTP Response
```

NestJS sẽ bắt lỗi và trả Response tương ứng.

5.3 Cơ chế xử lý Exception mặc định

Ví dụ:

```
@Get()
findAll() {

  throw new HttpException(
    'Forbidden',
    HttpStatus.FORBIDDEN
  );

}
```

Response:

```
{
  "statusCode": 403,
  "message": "Forbidden"
}
```

NestJS cung cấp sẵn cơ chế xử lý Exception.

Developer không cần tự tạo try/catch cho mọi Endpoint.

5.4 HttpException

Exception phổ biến nhất trong NestJS là:

HttpException

Ví dụ:

```
throw new HttpException(
  'User not found',
```

```

    HttpStatus.NOT_FOUND
);
Response:
{
  "statusCode": 404,
  "message": "User not found"
}

```

5.5 Cấu trúc của HttpException

Constructor:

```

new HttpException(
  response,
  status
)

```

Ví dụ:

```

throw new HttpException(
  'Invalid Request',
  HttpStatus.BAD_REQUEST
);

```

Trong đó:

Tham số	Ý nghĩa
response	Nội dung lỗi
status	HTTP Status Code

5.6 Các Built-in Exception

NestJS cung cấp nhiều Exception dựng sẵn.

BadRequestException

```

throw new BadRequestException();

```

Response:

400 Bad Request

UnauthorizedException

```
throw new UnauthorizedException();
```

Response:

401 Unauthorized

ForbiddenException

```
throw new ForbiddenException();
```

Response:

403 Forbidden

NotFoundException

```
throw new NotFoundException();
```

Response:

404 Not Found

ConflictException

```
throw new ConflictException();
```

Response:

409 Conflict

InternalServerErrorException

```
throw new InternalServerErrorException();
```

Response:

500 Internal Server Error

5.7 Trả thông báo lỗi tùy chỉnh

Ví dụ:

```
throw new NotFoundException(  
    'User does not exist'  
);  
Response:  
{  
    "statusCode": 404,  
    "message": "User does not exist",  
    "error": "Not Found"  
}
```

5.8 Trả về object lỗi tùy chỉnh

Ví dụ:

```
throw new HttpException(  
    {  
        code: 'USER_NOT_FOUND',  
        message: 'User does not exist'  
    },  
    HttpStatus.NOT_FOUND  
);  
Response:  
{  
    "code": "USER_NOT_FOUND",  
    "message": "User does not exist"  
}
```

5.9 Tại sao cần Exception Filter?

Giả sử toàn hệ thống muốn trả lỗi theo format:

```
{  
    "success": false,  
    "message": "User not found",  
    "timestamp": "2026-06-19T10:00:00Z"  
}
```

Nếu xử lý thủ công:

```
throw new HttpException(...)
```

ở mọi nơi sẽ rất khó quản lý.

Exception Filter giúp xử lý tập trung.

5.10 Exception Filter là gì?

Exception Filter là thành phần chịu trách nhiệm bắt và xử lý Exception.

Ví dụ:

```
Controller
  ↓
Exception
  ↓
Exception Filter
  ↓
Response
```

Filter cho phép:

- Ghi log lỗi.
 - Chuẩn hóa Response.
 - Ẩn thông tin nhạy cảm.
 - Thống nhất format lỗi.
-

5.11 Tạo Exception Filter

Nest CLI:

```
nest generate filter http-exception
```

Hoặc:

```
nest g filter http-exception
```

5.12 Custom Exception Filter đầu tiên

Ví dụ:

```
import {
  ExceptionFilter,
  Catch,
  ArgumentsHost,
  HttpException
} from '@nestjs/common';

@Catch(HttpException)
export class HttpExceptionFilter
  implements ExceptionFilter {
```

```

    catch(
        exception: HttpException,
        host: ArgumentsHost
    ) {

        const ctx =
            host.switchToHttp();

        const response =
            ctx.getResponse();

        const request =
            ctx.getRequest();

        response.status(
            exception.getStatus()
        ).json({
            timestamp:
                new Date().toISOString(),
            path: request.url
        });

    }
}

```

5.13 Decorator @Catch()

Decorator:

`@Catch()`

xác định loại Exception cần bắt.

Ví dụ:

`@Catch(HttpException)`

chỉ bắt:

`HttpException`

Bắt nhiều loại:

`@Catch(
 NotFoundException,`

```
ForbiddenException  
)
```

5.14 ArgumentsHost

ArgumentsHost cho phép truy cập Context hiện tại.

Ví dụ:

```
host.switchToHttp();
```

Trả về HTTP Context.

Lấy Request:

```
const request =  
  ctx.getRequest();
```

Lấy Response:

```
const response =  
  ctx.getResponse();
```

5.15 Áp dụng Filter cho Method

Ví dụ:

```
@Get('/:id')  
@UseFilters(  
  HttpExceptionHandler  
)  
findOne() {}
```

Filter chỉ áp dụng cho Endpoint đó.

5.16 Áp dụng Filter cho Controller

Ví dụ:

```
@UseFilters(  
  HttpExceptionHandler  
)  
@Controller('users')  
export class UsersController {}
```

Toàn bộ Endpoint trong Controller sẽ sử dụng Filter.

5.17 Global Exception Filter

Đăng ký trong:

```
main.ts  
  
app.useGlobalFilters(  
  new HttpExceptionFilter()  
);
```

Filter áp dụng cho toàn ứng dụng.

5.18 Catch tất cả Exception

Ví dụ:

```
@Catch()  
export class AllExceptionsFilter  
  implements ExceptionFilter {}
```

Filter sẽ bắt mọi Exception.

Ví dụ:

- Error
 - HttpException
 - TypeError
 - Database Error
-

5.19 Logging Exception

Ví dụ:

```
catch(  
  exception,  
  host  
) {  
  
  console.error(exception);  
  
}
```

Thường dùng để:

- Debug
 - Audit
 - Monitoring
-

5.20 Chuẩn hóa Error Response

Ví dụ:

```
{
  "success": false,
  "statusCode": 404,
  "message": "User not found",
  "timestamp": "2026-06-19T10:00:00Z"
}
```

Mọi lỗi trong hệ thống đều trả về cùng một format.

Đây là cách triển khai phổ biến trong các dự án thực tế.

5.21 Exception trong Service

Ví dụ:

```
@Injectable()
export class UsersService {

  findOne(id: number) {

    throw new NotFoundException(
      'User not found'
    );

  }

}
```

Controller:

```
@Get('/:id')
findOne(
  @Param('id')
  id: string
) {

  return this.usersService.findOne(
    Number(id)
  )
}
```

```
);  
  
}
```

Exception sẽ tự động được chuyển tới Filter.

5.22 Best Practices

Ném Exception từ Service

Tốt:

```
throw new NotFoundException(  
    'User not found'  
);
```

trong Service.

Không nên xử lý lỗi thủ công trong Controller.

Chuẩn hóa Error Response

Nên có một format lỗi thống nhất.

Ví dụ:

```
{  
  "success": false,  
  "message": "",  
  "statusCode": 400  
}
```

Sử dụng Built-in Exception

Ưu tiên:

```
NotFoundException  
BadRequestException  
ConflictException
```

thay vì:

```
HttpException(...)
```

trong các trường hợp phổ biến.

Sử dụng Global Filter

Các hệ thống lớn thường sử dụng:

```
app.useGlobalFilters(...)
```

để quản lý lỗi tập trung.

5.23 Common Mistakes

Trả lỗi bằng return

Sai:

```
return {  
  error: true  
};
```

Nên:

```
throw new BadRequestException();
```

Sử dụng Error thay vì HttpException

Sai:

```
throw new Error(  
  'User not found'  
);
```

Nên:

```
throw new NotFoundException(  
  'User not found'  
);
```

Tạo nhiều format lỗi khác nhau

Sai:

```
{  
  "error": true  
}
```

và

```
{  
  "status": "failed"  
}
```

trong cùng hệ thống.

Xử lý lỗi ở mọi Controller

Không nên:

```
try {  
}  
catch {  
}
```

lặp lại ở mọi Endpoint.

Hãy sử dụng Filter.

5.24 Tóm tắt chương

Trong chương này chúng ta đã tìm hiểu:

- Exception
- HttpException
- Built-in Exceptions
- Exception Filters
- @Catch()
- ArgumentsHost
- Global Filters
- Error Response Standardization

Exception Filter là cơ chế giúp NestJS xử lý lỗi tập trung, nhất quán và dễ bảo trì trong các ứng dụng từ nhỏ đến lớn.

Chương 6. Pipes

6.1 Mục tiêu chương học

Sau khi hoàn thành chương này, bạn sẽ hiểu:

- Pipe là gì trong NestJS.
- Pipe hoạt động ở vị trí nào trong Request Lifecycle.
- Vai trò của Validation và Transformation.
- Các Built-in Pipe của NestJS.
- Cách tạo Custom Pipe.

- Cách sử dụng ValidationPipe trong dự án thực tế.
 - Best Practices khi làm việc với DTO và Validation.
-

6.2 Pipe là gì?

Pipe là một thành phần dùng để:

- Chuyển đổi dữ liệu (Transformation)
- Kiểm tra dữ liệu (Validation)

trước khi dữ liệu được truyền vào Controller.

Ví dụ:

```
Client
  ↓
Request
  ↓
Pipe
  ↓
Controller
```

Pipe cho phép đảm bảo dữ liệu đầu vào luôn đúng định dạng trước khi xử lý.

6.3 Pipe trong Request Lifecycle

Một Request thường đi qua:

```
Client
  ↓
Middleware
  ↓
Guards
  ↓
Interceptors (Before)
  ↓
Pipes
  ↓
Controller
  ↓
Provider
  ↓
Interceptors (After)
  ↓
Response
```

Pipe được thực thi ngay trước Controller.

Điều này giúp Controller nhận được dữ liệu đã được kiểm tra.

6.4 Pipe làm được gì?

Pipe có hai nhiệm vụ chính.

Validation

Kiểm tra dữ liệu đầu vào.

Ví dụ:

```
@Get('/:id')
findOne(
  @Param('id', ParseIntPipe)
  id: number
) {}
```

Nếu:

GET /users/abc

NestJS trả:

400 Bad Request

Transformation

Chuyển đổi kiểu dữ liệu.

Ví dụ:

GET /users/100

Ban đầu:

"100"

Sau ParseIntPipe:

100

6.5 Built-in Pipes

NestJS cung cấp nhiều Pipe dựng sẵn.

Ví dụ:

ParseIntPipe
ParseBoolPipe
ParseUUIDPipe
ParseArrayPipe
ValidationPipe
DefaultValuePipe

Các Pipe này đã đáp ứng phần lớn nhu cầu thông thường.

6.6 ParseIntPipe

Dùng để chuyển String thành Number.

Ví dụ:

```
@Get('/:id')
findOne(
  @Param('id', ParseIntPipe)
  id: number
) {
  return id;
}
```

Request:

GET /users/100

Controller nhận:

100

Nếu:

GET /users/abc

NestJS trả:

400 Bad Request

6.7 ParseBoolPipe

Chuyển String thành Boolean.

Ví dụ:

```
@Get()
find(
  @Query(
```

```

        'active',
        ParseBoolPipe
    )
    active: boolean
) {
    return active;
}
Request:
GET /users?active=true
Kết quả:
true

```

6.8 ParseUUIDPipe

Kiểm tra UUID hợp lệ.

Ví dụ:

```

@Get('/:id')
findOne(
  @Param(
    'id',
    ParseUUIDPipe
  )
  id: string
) {}

```

UUID hợp lệ:

550e8400-e29b-41d4-a716-446655440000

UUID không hợp lệ:

abc

NestJS trả:

400 Bad Request

6.9 DefaultValuePipe

Thiết lập giá trị mặc định.

Ví dụ:

```

@Get()
findAll(
  @Query(
    'page',
    new DefaultValuePipe(1),
    ParseIntPipe
  )
  page: number
) {
  return page;
}

```

Request:

GET /users

Kết quả:

1

6.10 ParseArrayPipe

Dùng để xử lý mảng.

Ví dụ:

```

@Get()
findAll(
  @Query(
    'ids',
    new ParseArrayPipe({
      items: Number,
      separator: ',',
    })
  )
  ids: number[]
) {
  return ids;
}

```

Request:

GET /users?ids=1,2,3

Kết quả:

[1,2,3]

6.11 ValidationPipe

ValidationPipe là Pipe được sử dụng nhiều nhất trong NestJS.

Nó cho phép:

- Kiểm tra dữ liệu đầu vào.
- Tự động validate DTO.
- Loại bỏ dữ liệu không hợp lệ.

Ví dụ:

```
@Post()  
create(  
  @Body(  
    ValidationPipe  
  )  
  dto: CreateUserDto  
) {}
```

6.12 DTO là gì?

DTO (Data Transfer Object) là đối tượng mô tả dữ liệu đầu vào.

Ví dụ:

```
export class CreateUserDto {  
  
  name: string;  
  
  email: string;  
  
}
```

DTO giúp:

- Chuẩn hóa dữ liệu.
 - Dễ validate.
 - Dễ bảo trì.
-

6.13 Cài đặt Validation

ValidationPipe thường sử dụng:

```
npm install class-validator  
npm install class-transformer
```

Hai package này được NestJS sử dụng để validate DTO.

6.14 Validation Decorators

Ví dụ:

```
import {
  IsEmail,
  IsString
} from 'class-validator';
```

DTO:

```
export class CreateUserDto {

  @IsString()
  name: string;

  @IsEmail()
  email: string;

}
```

6.15 Sử dụng ValidationPipe

Ví dụ:

```
@Post()
create(
  @Body(
    new ValidationPipe()
  )
  dto: CreateUserDto
) {
  return dto;
}
```

Request hợp lệ:

```
{
  "name": "John",
  "email": "john@example.com"
}
```

Controller được gọi.

Request không hợp lệ:

```
{
  "name": 123,
  "email": "abc"
}
```

NestJS trả:

400 Bad Request

6.16 Global ValidationPipe

Thông thường ValidationPipe được cấu hình toàn cục.

main.ts:

```
app.useGlobalPipes(
  new ValidationPipe()
);
```

Sau đó toàn bộ Controller đều được validate.

6.17 Whitelist

Ví dụ:

```
app.useGlobalPipes(
  new ValidationPipe({
    whitelist: true
  })
);
```

DTO:

```
export class CreateUserDto {

  @IsString()
  name: string;

}
```

Request:

```
{
  "name": "John",
  "role": "admin"
}
```

Kết quả:

```
{  
  "name": "John"  
}
```

Field:

role

bị loại bỏ.

6.18 ForbidNonWhitelisted

Ví dụ:

```
new ValidationPipe({  
  whitelist: true,  
  forbidNonWhitelisted: true  
})
```

Request:

```
{  
  "name": "John",  
  "role": "admin"  
}
```

NestJS trả:

400 Bad Request

6.19 Transform

Cho phép tự động chuyển kiểu dữ liệu.

```
new ValidationPipe({  
  transform: true  
})
```

Ví dụ:

```
@Get('/:id')  
findOne(  
  @Param('id')  
  id: number  
) {}
```

Request:

GET /users/100

NestJS tự động chuyển:

"100"

thành:

100

6.20 Tạo Custom Pipe

Ví dụ:

```
import {
  PipeTransform,
  Injectable,
  BadRequestException
} from '@nestjs/common';

@Injectable()
export class PositiveNumberPipe
  implements PipeTransform {

  transform(value: any) {

    if (value <= 0) {

      throw new BadRequestException(
        'Value must be positive'
      );

    }

    return value;

  }

}
```

6.21 Sử dụng Custom Pipe

Ví dụ:

```
@Get('/:id')
findOne(
```

```

    @Param(
        'id',
        PositiveNumberPipe
    )
    id: number
) {}

```

Request:

GET /users/-1

Kết quả:

400 Bad Request

6.22 Pipe Scope

Pipe có thể áp dụng ở nhiều mức.

Parameter Level

```

@Param(
    'id',
    ParseIntPipe
)

```

Method Level

```

@UsePipes(
    ValidationPipe
)

```

Controller Level

```

@UsePipes(
    ValidationPipe
)
@Controller('users')

```

Global Level

```

app.useGlobalPipes(...)

```

6.23 Best Practices

Luôn sử dụng DTO

Không nên:

```
@Body() body: any
```

Nên:

```
@Body()  
dto: CreateUserDto
```

Sử dụng ValidationPipe toàn cục

Ví dụ:

```
app.useGlobalPipes(  
  new ValidationPipe()  
);
```

Bật Whitelist

Ví dụ:

```
whitelist: true
```

để loại bỏ dữ liệu không mong muốn.

Không đặt Validation trong Service

Validation nên được thực hiện ở Pipe.

Service tập trung vào Business Logic.

6.24 Common Mistakes

Không sử dụng DTO

Sai:

```
@Body() data: any
```

Không bật ValidationPipe

Khi đó mọi dữ liệu đều đi vào Controller mà không được kiểm tra.

Validate thủ công trong Controller

Sai:

```
if (!body.email) {  
    throw ...  
}
```

Nên sử dụng:

ValidationPipe

Đặt Business Logic trong Pipe

Pipe chỉ nên:

- Validate
- Transform

Không nên:

```
query database  
send email  
create order
```

6.25 Tóm tắt chương

Trong chương này chúng ta đã tìm hiểu:

- Pipe
- Built-in Pipes
- ParseIntPipe
- ParseBoolPipe
- ParseUUIDPipe
- DefaultValuePipe
- ParseArrayPipe
- ValidationPipe
- DTO
- class-validator
- class-transformer
- Custom Pipes
- Pipe Scope

Pipe là cơ chế quan trọng giúp NestJS kiểm tra và chuyển đổi dữ liệu trước khi Controller xử lý Request, góp phần xây dựng các API an toàn, nhất quán và dễ bảo trì.

Chương 7. Guards

7.1 Mục tiêu chương học

Sau khi hoàn thành chương này, bạn sẽ hiểu:

- Guard là gì trong NestJS.
 - Guard hoạt động ở vị trí nào trong Request Lifecycle.
 - Sự khác biệt giữa Middleware và Guard.
 - Cách tạo Guard.
 - Cách áp dụng Guard cho Route, Controller và toàn ứng dụng.
 - Authentication và Authorization trong NestJS.
 - Cách sử dụng Metadata và Reflector.
 - Best Practices khi xây dựng Guard.
-

7.2 Guard là gì?

Guard là thành phần quyết định một Request có được phép tiếp tục xử lý hay không.

Guard hoạt động như một cổng kiểm tra quyền truy cập.

Ví dụ:

```
Client
  ↓
Guard
  ↓
Controller
```

Nếu Guard cho phép:

`true`

Request tiếp tục.

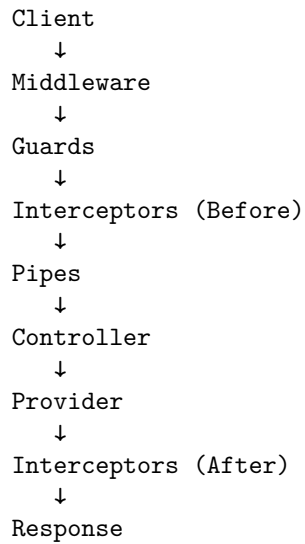
Nếu Guard từ chối:

`false`

Request bị chặn.

7.3 Guard trong Request Lifecycle

Một Request thường đi qua:



Guard chạy sau Middleware và trước Controller.

7.4 Guard dùng để làm gì?

Các trường hợp phổ biến:

Authentication

Kiểm tra người dùng đã đăng nhập chưa.

Ví dụ:

Authorization: Bearer token

Authorization

Kiểm tra người dùng có quyền truy cập hay không.

Ví dụ:

Admin
User
Manager

Feature Access

Kiểm tra tính năng được bật hay chưa.

Ví dụ:

Premium Feature

Beta Feature

7.5 Tại sao không dùng Middleware?

Middleware có thể đọc Request.

Tuy nhiên Middleware không biết Route hiện tại yêu cầu quyền gì.

Ví dụ:

GET /users

và

DELETE /users

có thể yêu cầu quyền khác nhau.

Guard được thiết kế để giải quyết vấn đề này.

7.6 Tạo Guard

Nest CLI:

```
nest generate guard auth
```

Hoặc:

```
nest g gu auth
```

Kết quả:

```
src/
  auth/
    auth.guard.ts
    auth.guard.spec.ts
```

7.7 CanActivate Interface

Mọi Guard đều triển khai:

CanActivate

Ví dụ:

```
import {
  CanActivate,
  ExecutionContext
} from '@nestjsjs/common';

export class AuthGuard
  implements CanActivate {

  canActivate(
    context: ExecutionContext
  ): boolean {

    return true;

  }
}
```

7.8 Phương thức canActivate()

Đây là phương thức quan trọng nhất của Guard.

Ví dụ:

```
canActivate(
  context: ExecutionContext
): boolean {

  return true;

}
```

Nếu:

true

Request tiếp tục.

Nếu:

false

NestJS trả:

403 Forbidden

7.9 Guard đơn giản đầu tiên

Ví dụ:

```
@Injectable()
export class AuthGuard
  implements CanActivate {

  canActivate(
    context: ExecutionContext
  ): boolean {

    return false;

  }

}
```

Mọi Request sẽ bị từ chối.

Response:

403 Forbidden

7.10 Áp dụng Guard cho Route

Ví dụ:

```
@Get()
@UseGuards(AuthGuard)
findAll() {}
```

Request sẽ phải đi qua:

AuthGuard

trước khi tới Controller.

7.11 Áp dụng Guard cho Controller

Ví dụ:

```
@Controller('users')
@UseGuards(AuthGuard)
export class UsersController {}
```

Tất cả Endpoint trong Controller đều sử dụng Guard.

7.12 Global Guard

main.ts:

```
app.useGlobalGuards(  
  new AuthGuard()  
);
```

Guard sẽ áp dụng cho toàn bộ hệ thống.

7.13 ExecutionContext

Guard sử dụng:

ExecutionContext

để truy cập Request hiện tại.

Ví dụ:

```
canActivate(  
  context: ExecutionContext  
)
```

Lấy Request:

```
const request =  
  context  
    .switchToHttp()  
    .getRequest();
```

7.14 Đọc Header trong Guard

Ví dụ:

```
canActivate(  
  context: ExecutionContext  
): boolean {  
  
  const request =  
    context  
      .switchToHttp()  
      .getRequest();  
  
  const token =  
    request.headers.authorization;
```

```
        return !!token;
    }
}
```

Nếu không có token:

403 Forbidden

7.15 Authentication Guard

Ví dụ:

```
@Injectable()
export class AuthGuard
    implements CanActivate {

    canActivate(
        context: ExecutionContext
    ): boolean {

        const request =
            context
                .switchToHttp()
                .getRequest();

        const token =
            request.headers.authorization;

        return !!token;
    }
}
```

Request:

GET /users

Authorization: Bearer abc

Được phép truy cập.

7.16 Authorization là gì?

Authentication trả lời câu hỏi:

Bạn là ai?

Authorization trả lời câu hỏi:

Bạn được phép làm gì?

Ví dụ:

Admin
Editor
User

7.17 Role-based Authorization

Giả sử:

```
request.user = {  
  id: 1,  
  role: 'admin'  
};
```

Guard:

```
canActivate(  
  context: ExecutionContext  
)< boolean {  
  
  const request =  
    context  
      .switchToHttp()  
      .getRequest();  
  
  return request.user.role === 'admin';  
}
```

7.18 Metadata

NestJS cho phép gắn Metadata vào Route.

Ví dụ:

```
@SetMetadata(  
  'roles',  
  ['admin']  
)
```

Controller:

```
@Get()
@SetMetadata(
  'roles',
  ['admin']
)
findAll() {}
```

7.19 Reflector

Reflector giúp đọc Metadata.

Ví dụ:

```
constructor(
  private reflector: Reflector
) {}
```

Lấy Metadata:

```
const roles =
  this.reflector.get(
    'roles',
    context.getHandler()
  );
```

7.20 Tạo Decorator Roles

Ví dụ:

```
import { SetMetadata }
from '@nestjs/common';

export const Roles =
  (...roles: string[]) =>
    SetMetadata(
      'roles',
      roles
    );
```

Sử dụng:

```
@Roles('admin')
```

7.21 Roles Guard

Ví dụ:

```
@Injectable()
export class RolesGuard
  implements CanActivate {

  constructor(
    private reflector: Reflector
  ) {}

  canActivate(
    context: ExecutionContext
  ): boolean {

    const roles =
      this.reflector.get(
        'roles',
        context.getHandler()
      );

    if (!roles) {
      return true;
    }

    const request =
      context
        .switchToHttp()
        .getRequest();

    return roles.includes(
      request.user.role
    );

  }
}
```

7.22 Guard trả Exception

Thay vì:

```
return false;
```

có thể:

```
throw new ForbiddenException();
```

Điều này cho phép tùy chỉnh lỗi.

7.23 Nhiều Guards

Ví dụ:

```
@UseGuards(  
    AuthGuard,  
    RolesGuard  
)
```

Thứ tự:

```
AuthGuard  
    ↓  
RolesGuard  
    ↓  
Controller
```

Tất cả Guard phải thành công.

7.24 Guard Scope

Guard có thể áp dụng ở:

Method Level

```
@UseGuards(AuthGuard)
```

Controller Level

```
@Controller()  
@UseGuards(AuthGuard)
```

Global Level

```
app.useGlobalGuards(...)
```

7.25 APP_GUARD

Một cách phổ biến để đăng ký Global Guard:

```
@Module({
  providers: [
    {
      provide: APP_GUARD,
      useClass: AuthGuard
    }
  ]
})
```

Ưu điểm:

- Hỗ trợ Dependency Injection.
 - Có thể inject Service.
-

7.26 Best Practices

Authentication bằng Guard

Không nên:

```
if (!token)
```

trong Controller.

Nên sử dụng Guard.

Authorization bằng Guard

Role checking nên nằm trong Guard.

Không nên đặt trong Service hoặc Controller.

Sử dụng Decorator tùy chỉnh

Tốt:

```
@Roles('admin')
```

Thay vì:

```
@SetMetadata(...)
```

ở mọi nơi.

Tách Authentication và Authorization

Không nên:

`AuthAndRoleGuard`

quá lớn.

Nên:

`AuthGuard`

`RolesGuard`

riêng biệt.

7.27 Common Mistakes

Dùng Middleware để phân quyền

Sai:

```
if (role !== 'admin')
```

trong Middleware.

Đặt Role Logic trong Controller

Sai:

```
if (  
  request.user.role  
  !== 'admin'  
)
```

trong Controller.

Tạo Global Guard không cần thiết

Không phải mọi Guard đều nên Global.

Một số Guard chỉ nên áp dụng cho các Route đặc biệt.

Không sử dụng Metadata

Điều này khiến Guard khó tái sử dụng.

7.28 So sánh Middleware và Guard

Middleware	Guard
Chạy trước Guard	Chạy sau Middleware
Không biết Route Metadata	Biết Route Metadata
Logging	Authorization
Request Tracking	Authentication
Correlation ID	Role Checking

7.29 Tóm tắt chương

Trong chương này chúng ta đã tìm hiểu:

- Guards
- CanActivate
- ExecutionContext
- Authentication
- Authorization
- Metadata
- Reflector
- Roles Guard
- Global Guard
- APP_GUARD

Guard là cơ chế kiểm soát quyền truy cập mạnh mẽ của NestJS và là thành phần trung tâm trong các hệ thống Authentication và Authorization hiện đại.

Chương 8. Interceptors

8.1 Mục tiêu chương học

Sau khi hoàn thành chương này, bạn sẽ hiểu:

- Interceptor là gì trong NestJS.
- Interceptor hoạt động ở vị trí nào trong Request Lifecycle.
- Cách tạo Interceptor.
- Cách xử lý dữ liệu trước và sau Controller.
- Cách biến đổi Response.
- Cách ghi log và đo thời gian xử lý.
- Cách sử dụng Interceptor trong các dự án thực tế.
- Best Practices khi xây dựng Interceptor.

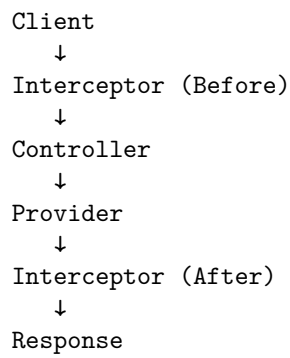
8.2 Interceptor là gì?

Interceptor là thành phần cho phép can thiệp vào quá trình xử lý Request và Response.

Khác với Middleware chỉ xử lý trước Controller, Interceptor có thể:

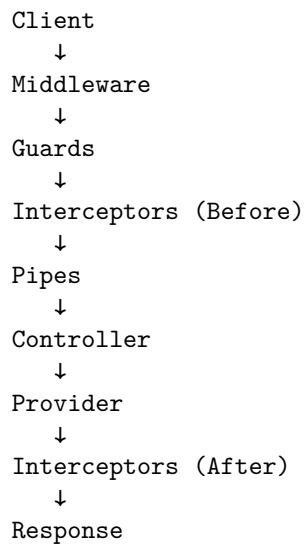
- Thực thi trước Controller.
- Thực thi sau Controller.
- Thay đổi dữ liệu trả về.
- Bắt và xử lý luồng dữ liệu.

Ví dụ:



8.3 Vị trí của Interceptor trong Request Lifecycle

Một Request hoàn chỉnh:



Interceptor là thành phần duy nhất có khả năng hoạt động ở cả hai chiều.

8.4 Interceptor dùng để làm gì?

Các trường hợp phổ biến:

Logging

Request Log
Response Log

Response Mapping

Biến đổi dữ liệu trả về.

Caching

Lưu cache Response.

Timeout

Giới hạn thời gian xử lý Request.

Audit Log

Theo dõi hoạt động người dùng.

Performance Monitoring

Đo thời gian xử lý.

8.5 Tạo Interceptor

Nest CLI:

```
nest generate interceptor logging
```

Hoặc:

```
nest g interceptor logging
```

Kết quả:

```
src/  
  logging/  
    logging.interceptor.ts  
    logging.interceptor.spec.ts
```

8.6 NestInterceptor Interface

Mọi Interceptor đều triển khai:

NestInterceptor

Ví dụ:

```
import {  
  Injectable,  
  NestInterceptor,  
  ExecutionContext,  
  CallHandler  
} from '@nestjs/common';  
  
@Injectable()  
export class LoggingInterceptor  
  implements NestInterceptor {  
  
  intercept(  
    context: ExecutionContext,  
    next: CallHandler  
  ) {  
  
    return next.handle();  
  
  }  
}
```

8.7 Phương thức intercept()

Đây là phương thức quan trọng nhất.

```
intercept(  
  context: ExecutionContext,
```

```
    next: CallHandler
  )
```

Hai thành phần chính:

Thành phần	Vai trò
ExecutionContext	Thông tin Request hiện tại
CallHandler	Tiếp tục luồng xử lý

8.8 CallHandler

```
next.handle()
```

đại diện cho phần xử lý tiếp theo.

Ví dụ:

```
intercept(
  context,
  next
) {

  console.log('Before');

  return next.handle();

}
```

Luồng:

Before
↓
Controller

8.9 Interceptor trước Controller

Ví dụ:

```
intercept(
  context,
  next
) {

  console.log(
    'Before Controller'
  )

  next.handle()
}
```

```

    );

    return next.handle();
}

```

Kết quả:

Before Controller
Controller

8.10 Interceptor sau Controller

Interceptor sử dụng RxJS để xử lý dữ liệu trả về.

Ví dụ:

```

import { tap } from 'rxjs/operators';

intercept(
  context,
  next
) {

  return next.handle().pipe(
    tap(() => {
      console.log(
        'After Controller'
      );
    })
  );
}

```

Luồng:

Controller
↓
After Controller

8.11 Logging Interceptor

Ví dụ:

```

@Injectable()
export class LoggingInterceptor

```

```

implements NestInterceptor {

  intercept(
    context,
    next
  ) {

    console.log(
      'Incoming Request'
    );

    return next.handle();

  }
}

```

8.12 Đo thời gian xử lý Request

Ví dụ:

```

intercept(
  context,
  next
) {

  const now = Date.now();

  return next.handle().pipe(
    tap(() => {

      console.log(
        Date.now() - now
      );

    })
  );

}

```

Kết quả:

```

35 ms
120 ms
15 ms

```

8.13 Response Mapping

Controller:

```
@Get()
findAll() {

    return {
        id: 1,
        name: 'John'
    };

}
```

Interceptor:

```
intercept(
    context,
    next
) {

    return next.handle().pipe(
        map(data => ({
            success: true,
            data
        })))
    );

}
```

Response:

```
{
  "success": true,
  "data": {
    "id": 1,
    "name": "John"
  }
}
```

8.14 Loại bỏ dữ liệu nhạy cảm

Ví dụ:

Controller:

```
return {  
  id: 1,  
  username: 'john',  
  password: '123456'  
};
```

Interceptor:

```
map(user => {  
  
  delete user.password;  
  
  return user;  
  
})
```

Response:

```
{  
  "id": 1,  
  "username": "john"  
}
```

8.15 Transform dữ liệu

Ví dụ:

```
map(data => {  
  
  return {  
    result: data  
  };  
  
})
```

Response:

```
{  
  "result": {  
    ...  
  }  
}
```

8.16 Exception Mapping

Interceptor có thể xử lý lỗi.

Ví dụ:

```
import {
  catchError
} from 'rxjs/operators';
return next.handle().pipe(
  catchError(error => {

    throw error;

  })
);
```

Tuy nhiên xử lý lỗi tập trung thường nên dùng Exception Filter.

8.17 Timeout Interceptor

Ví dụ:

```
import { timeout } from 'rxjs/operators';

return next.handle().pipe(
  timeout(5000)
);
```

Nếu Request vượt quá:

5000 ms

sẽ phát sinh lỗi.

8.18 Cache Interceptor

NestJS cung cấp:

CacheInterceptor

Ví dụ:

```
@UseInterceptors(
  CacheInterceptor
)
```

Response có thể được lưu cache để tăng hiệu năng.

8.19 File Upload Interceptor

NestJS cung cấp:

`FileInterceptor`

Ví dụ:

```
@Post('upload')
@UseInterceptors(
  FileInterceptor('file')
)
uploadFile() {}
```

Interceptor này được sử dụng rất nhiều trong các API upload file.

8.20 Áp dụng Interceptor cho Method

Ví dụ:

```
@Get()
@UseInterceptors(
  LoggingInterceptor
)
findAll() {}
```

8.21 Áp dụng Interceptor cho Controller

Ví dụ:

```
@Controller('users')
@UseInterceptors(
  LoggingInterceptor
)
export class UsersController {}
```

Tất cả Endpoint trong Controller sẽ sử dụng Interceptor.

8.22 Global Interceptor

main.ts:

```
app.useGlobalInterceptors(
  new LoggingInterceptor()
);
```

Interceptor áp dụng cho toàn ứng dụng.

8.23 APP_INTERCEPTOR

Một cách phổ biến:

```
@Module({
  providers: [
    {
      provide: APP_INTERCEPTOR,
      useClass: LoggingInterceptor
    }
  ]
})
export class AppModule {}
```

Ưu điểm:

- Hỗ trợ Dependency Injection.
 - Có thể inject Service.
-

8.24 ExecutionContext

Interceptor cũng sử dụng:

ExecutionContext

Ví dụ:

```
const request =
  context
    .switchToHttp()
    .getRequest();
```

Có thể lấy:

- Request
 - Response
 - User
 - Headers
-

8.25 RxJS trong Interceptor

Phần lớn sức mạnh của Interceptor đến từ RxJS.

Các Operator phổ biến:

```
map()
tap()
catchError()
timeout()
```

Ví dụ:

```
next.handle().pipe(
  map(...)
)
```

8.26 So sánh Middleware và Interceptor

Middleware	Interceptor
Chỉ xử lý trước Controller	Trước và sau Controller
Không xử lý Response	Có thể xử lý Response
Logging Request	Logging Request và Response
Không thay đổi dữ liệu trả về	Có thể thay đổi Response

8.27 So sánh Guard và Interceptor

Guard	Interceptor
Quyết định quyền truy cập	Xử lý luồng dữ liệu
Authentication	Response Mapping
Authorization	Logging
Role Checking	Caching

8.28 Best Practices

Không đặt Business Logic trong Interceptor

Sai:

```
createOrder()
sendEmail()
updateDatabase()
```

Interceptor không nên chứa nghiệp vụ.

Dùng Interceptor cho Response Format

Ví dụ:

```
{  
  "success": true,  
  "data": {}  
}
```

Toàn hệ thống nên thống nhất.

Sử dụng Global Interceptor cho Logging

Ví dụ:

APP_INTERCEPTOR

để ghi log tập trung.

Giữ Interceptor đơn giản

Một Interceptor nên có một trách nhiệm chính.

Ví dụ:

LoggingInterceptor

TransformInterceptor

TimeoutInterceptor

8.29 Common Mistakes

Dùng Interceptor để Authorization

Sai:

```
if (role !== 'admin')
```

Nên sử dụng Guard.

Dùng Interceptor để Validation

Sai:

```
if (!email)
```

Nên sử dụng Pipe.

Quá nhiều logic trong một Interceptor

Sai:

```
Logging  
Caching  
Validation  
Authorization
```

trong cùng một class.

Không sử dụng RxJS Operators

Interceptor sẽ mất đi phần lớn sức mạnh nếu chỉ gọi:

```
next.handle();
```

mà không tận dụng:

```
map()  
tap()  
catchError()
```

8.30 Tóm tắt chương

Trong chương này chúng ta đã tìm hiểu:

- Interceptor
- NestInterceptor
- ExecutionContext
- CallHandler
- RxJS Operators
- Logging Interceptor
- Transform Interceptor
- Timeout Interceptor
- Cache Interceptor
- File Interceptor
- Global Interceptor
- APP_INTERCEPTOR

Interceptor là một trong những tính năng mạnh mẽ nhất của NestJS, cho phép can thiệp vào toàn bộ luồng xử lý Request và Response, đồng thời hỗ trợ các tác vụ như Logging, Monitoring, Response Transformation và Caching một cách tập trung và nhất quán.

Chương 9. Custom Decorators

9.1 Mục tiêu chương học

Sau khi hoàn thành chương này, bạn sẽ hiểu:

- Decorator là gì trong NestJS.
 - Vai trò của Decorator trong kiến trúc NestJS.
 - Metadata hoạt động như thế nào.
 - Cách tạo Custom Decorator.
 - Cách truyền dữ liệu thông qua Decorator.
 - Cách kết hợp Decorator với Guard.
 - Cách xây dựng các Decorator thường gặp trong dự án thực tế.
-

9.2 Decorator là gì?

Decorator là một cú pháp đặc biệt của TypeScript cho phép gắn Metadata vào:

- Class
- Method
- Property
- Parameter

Ví dụ:

```
@Controller('users')
export class UsersController {}
```

Ở đây:

```
@Controller()
```

chính là một Decorator.

9.3 Decorator trong NestJS

NestJS sử dụng Decorator rất nhiều.

Ví dụ:

```
@Controller()
```

```
@Get()
```

```
@Post()
```

```
@Injectable()
```

```
@Module()
```

`@Body()`

`@Param()`

`@UseGuards()`

Gần như toàn bộ NestJS được xây dựng dựa trên Decorator và Metadata.

9.4 Metadata là gì?

Metadata là dữ liệu mô tả bổ sung cho một đối tượng.

Ví dụ:

`@Controller('users')`

NestJS sẽ lưu:

```
{
  path: 'users'
}
```

như một Metadata.

Sau đó Framework đọc Metadata để xây dựng Routing.

9.5 Tại sao cần Custom Decorator?

Giả sử:

`request.user`

được gán bởi Authentication Guard.

Controller:

```
@Get('profile')
getProfile(
  @Req()
  request
) {

  return request.user;
}
```

Cách này hoạt động nhưng:

- Dài dòng.
- Phụ thuộc Express Request.

- Khó tái sử dụng.

Giải pháp:

```
@Get('profile')
getProfile(
  @CurrentUser()
  user
) {

  return user;
}
```

9.6 Tạo Decorator đầu tiên

NestJS cung cấp:

```
createParamDecorator()
```

Ví dụ:

```
import {
  createParamDecorator,
  ExecutionContext
} from '@nestjs/common';

export const CurrentUser =
  createParamDecorator(
    (
      data: unknown,
      context: ExecutionContext
    ) => {

      const request =
        context
          .switchToHttp()
          .getRequest();

      return request.user;

    }
  );
```

9.7 Sử dụng Custom Decorator

Ví dụ:

```
@Get('profile')
getProfile(
  @CurrentUser()
  user
) {

  return user;
}
```

Kết quả:

```
request.user
```

được inject trực tiếp vào Method.

9.8 ExecutionContext trong Decorator

Custom Decorator cũng sử dụng:

ExecutionContext

Ví dụ:

```
const request =
  context
    .switchToHttp()
    .getRequest();
```

Điều này cho phép truy cập:

- Request
 - Response
 - Headers
 - User
-

9.9 Truyền tham số vào Decorator

Ví dụ:

```
@CurrentUser('email')
```

Decorator:

```

export const CurrentUser =
  createParamDecorator(
    (
      property: string,
      context: ExecutionContext
    ) => {

      const request =
        context
          .switchToHttp()
          .getRequest();

      return request.user[property];

    }
  );

```

Controller:

```

@Get()
findMe(
  @CurrentUser('email')
  email: string
) {

  return email;

}

```

9.10 Decorator lấy User hiện tại

Ví dụ thực tế:

```

@Get('me')
getMe(
  @CurrentUser()
  user
) {

  return user;

}

```

Thay vì:

```
@Req()  
request
```

9.11 Decorator lấy User ID

Ví dụ:

```
export const UserId =  
  createParamDecorator(  
    (  
      data,  
      context  
    ) => {  
  
      const request =  
        context  
          .switchToHttp()  
          .getRequest();  
  
      return request.user.id;  
    }  
  );
```

Controller:

```
@Get()  
findMe(  
  @UserId()  
  userId: number  
) {  
  
  return userId;  
}
```

9.12 Decorator đọc Header

Ví dụ:

```
export const HeaderValue =  
  createParamDecorator(  
    (  

```

```

        key: string,
        context
    ) => {

        const request =
            context
                .switchToHttp()
                .getRequest();

        return request.headers[key];

    }
};

```

Sử dụng:

```

@Get()
find(
    @HeaderValue(
        'user-agent'
    )
    userAgent: string
) {}

```

9.13 Decorator đọc IP

Ví dụ:

```

export const ClientIp =
    createParamDecorator(
        (
            data,
            context
        ) => {

            const request =
                context
                    .switchToHttp()
                    .getRequest();

            return request.ip;

        }
    );

```

Controller:

```
@Get()
findAll(
  @ClientId()
  ip: string
) {}
```

9.14 Method Decorator

Ngoài Parameter Decorator, có thể tạo Method Decorator.

Ví dụ:

```
@Roles('admin')
```

Thực chất:

```
@Roles()
```

là một Custom Decorator.

9.15 SetMetadata

NestJS cung cấp:

```
SetMetadata()
```

để tạo Metadata.

Ví dụ:

```
import {
  SetMetadata
} from '@nestjs/common';
```

Decorator:

```
export const Roles =
  (...roles: string[]) =>
    SetMetadata(
      'roles',
      roles
    );
```

Sử dụng:

```
@Roles('admin')
```

9.16 Metadata được lưu ở đâu?

Ví dụ:

```
@Roles('admin')
```

NestJS lưu:

```
{  
  roles: ['admin']  
}
```

trên Method.

Guard có thể đọc lại thông tin này bằng Reflector.

9.17 Reflector

Ví dụ:

```
constructor(  
  private reflector: Reflector  
) {}
```

Đọc Metadata:

```
const roles =  
  this.reflector.get(  
    'roles',  
    context.getHandler()  
  );
```

9.18 Kết hợp Decorator với Guard

Decorator:

```
@Roles('admin')
```

Guard:

```
const roles =
  this.reflector.get(
    'roles',
    context.getHandler()
  );
```

Request:

Admin

→ Cho phép

User

→ Từ chối

Đây là mô hình Authorization phổ biến nhất trong NestJS.

9.19 Composite Decorator

Một Decorator có thể kết hợp nhiều Decorator khác.

NestJS cung cấp:

`applyDecorators()`

Ví dụ:

```
import {
  applyDecorators,
  UseGuards
} from '@nestjs/common';
```

```
export function AdminOnly() {

  return applyDecorators(
    Roles('admin'),
    UseGuards(
      AuthGuard,
      RolesGuard
    )
  );
}
```

Sử dụng:

```
@AdminOnly()  
@Get()  
findAll() {}
```

9.20 Decorator trong dự án thực tế

Các Decorator thường gặp:

```
@CurrentUser()
```

```
@UserId()
```

```
@Roles()
```

```
@Permissions()
```

```
@Public()
```

```
@AdminOnly()
```

9.21 Public Decorator

Ví dụ:

```
export const Public =  
  () =>  
    SetMetadata(  
      'isPublic',  
      true  
    );
```

Controller:

```
@Public()  
@Get('login')  
login() {}
```

Guard:

```
const isPublic =  
  this.reflector.get(  
    'isPublic',  
    context.getHandler()  
  );
```

Nếu:

```
true
```

bỏ qua Authentication.

9.22 Decorator Scope

Decorator có thể áp dụng cho:

Class

```
@Controller()
```

Method

```
@Get()
```

Parameter

```
@Body()
```

Property

```
@Inject()
```

9.23 Best Practices

Tạo Decorator cho logic lặp lại

Ví dụ:

```
@CurrentUser()
```

thay vì:

```
@Req()  
request
```

ở mọi Controller.

Đặt tên rõ ràng

Tốt:

```
@CurrentUser()  
@UserId()  
@Public()
```

Không nên:

```
@Data()  
@Info()
```

Kết hợp với Guard

Decorator nên mô tả quyền truy cập.

Guard thực hiện kiểm tra.

Giữ Decorator đơn giản

Decorator chủ yếu dùng để:

- Đọc dữ liệu.
- Lưu Metadata.

Không nên chứa Business Logic.

9.24 Common Mistakes

Đặt Business Logic trong Decorator

Sai:

```
query database
create order
send email
```

Decorator không nên thực hiện nghiệp vụ.

Không dùng Decorator khi logic lặp lại

Sai:

```
request.user
```

lặp lại trong hàng chục Controller.

Không sử dụng Metadata

Dẫn tới việc Guard khó tái sử dụng.

Decorator quá phức tạp

Một Decorator chỉ nên phục vụ một mục đích cụ thể.

9.25 Tóm tắt chương

Trong chương này chúng ta đã tìm hiểu:

- Decorators
- Metadata
- createParamDecorator()
- SetMetadata()
- Reflector
- Roles Decorator
- Public Decorator
- CurrentUser Decorator
- Composite Decorator
- applyDecorators()

Custom Decorators là một trong những cơ chế giúp NestJS trở nên linh hoạt và dễ mở rộng, cho phép đóng gói các hành vi lặp lại thành những thành phần có thể tái sử dụng trong toàn bộ ứng dụng.